RESEARCH ARTICLE

Doubly Nested Semantic Communication: Rate-Nested Token Transmission and Time-Nested Language-Model Memories for Drift-Resilient 6G Links

Abraheem Rashid[1] | Faisal Iradat[2] | Waseem Iqbal[3] | Yawar Abbas Bangash[4]

[1]Department of Computer Science, Brunel University London, London, United Kingdom

[2]Department of Computer Science, School of Mathematics and Science, Institute of Business Administration, Karachi, Pakistan

[3]Department of Electrical and Computer Engineering, Sultan Qaboos University, Muscat, Oman

[4]School of Computing and Artificial Intelligence, FET, Sunway University, Selangor Darul Ehsan, Malaysia

Correspondence: Waseem Iqbal, Department of Electrical and Computer Engineering, Sultan Qaboos University, Muscat, Oman. Email: m.waseem@squ.edu.om

Email addresses: 2546393@brunel.ac.uk (A. Rashid); firadat@iba.edu.pk (F. Iradat); m.waseem@squ.edu.om (W. Iqbal); yawarb@sunway.edu.my (Y. A. Bangash)

Abstract

Large language models (LLMs) enable a semantic receiver to recover text from few channel symbols, because a model shared by both ends predicts the transmitted text. After deployment, however, content drifts, and the air time needed to keep the receiver's knowledge aligned is rarely counted. This article proposes doubly nested semantic communication (DN-SC), a token-level transceiver nested both in rate and in time. Each token is transmitted as a prefix of a 16-symbol codeword whose shells are power-controlled separately, so that the receiver can request further symbols only for uncertain tokens; a frozen GPT-2 model serves as the prior of a Bayesian beam-search decoder; and three receiver memories (context, logit bias, adapter slots) adapt the prior at three timescales. The return-link requests and every synchronized bit are converted into channel symbols. On a hierarchical drift stream with six seeds (AWGN, mean signal-to-noise ratio of 10 dB), DN-SC attains 90% token accuracy with 7.08 real channel symbols per token, against 9.23 for a frozen fixed-rate receiver with the same decoder: a reduction of 23.3% with the requests included, observed on every seed, and of 27.9% (95% confidence interval 21.6–31.7%) under free-running decoding, the measurement selected by the pre-specified rule. The context memory, which requires no synchronization, provides about three quarters of the reduction in prior cross-entropy achieved by the memories; relative to it, the shipped memories repay their synchronization cost only when one update serves more than approximately 6100 messages, an indicative figure because both costs are upper bounds.

Keywords

6G | semantic communication | large language models | joint source–channel coding | incremental redundancy | continual adaptation | nested learning

1 | Introduction

The sixth generation (6G) of mobile networks is expected to support services whose value lies in the meaning of the delivered content rather than in the exact reproduction of every bit [1–4]. Exact reproduction is the problem formulated and solved by Shannon [5]. Semantic communication addresses the level above it, identified by Weaver [6], and designs transmitter and receiver jointly around the information that the receiver actually requires [7–9]. Learned joint source–channel coding (JSCC) has shown that such systems degrade gracefully as the channel deteriorates and can outperform separate source and channel coding at short block lengths, both for text [10, 11] and for images [12, 13].

Large language models (LLMs) contribute a further resource to this design space, namely an accurate statistical model of what a sender is likely to say. When both ends share such a model, the receiver can combine the likelihood of the received symbols with the model's prediction of the next token and decide on the most probable sentence, so that fewer channel symbols suffice for a given fidelity [14]. The context among tokens can be exploited in the same manner across modalities [15], pre-trained language models have been used to rank the semantic importance of words [16], and large AI models are being investigated as the knowledge base of semantic transceivers [17–19].

Two costs, however, are usually left outside the analysis of such systems. The first is the cost of keeping the shared knowledge aligned. After deployment the content changes, the topic of a conversation within minutes and the vocabulary of a domain over days, and a frozen receiver degrades accordingly. The receiver cannot adapt on its own, since only the transmitter has access to the source text; every update of the receiver's model must therefore be carried over the very link that the semantic codec is intended to economize. Existing work adapts or updates the knowledge of semantic transceivers after deployment [20–22], but for a language-model prior updated at several timescales the air time of the updates has not, to the authors' knowledge, been set against the rate that they save. The second is the cost of adapting the rate. Incremental redundancy allows a receiver to request additional symbols only when necessary [23, 24], yet every request travels on a return link, and a token-level policy issues many requests.

This article studies both costs within a single system, termed doubly nested semantic communication (DN-SC). The system is nested in rate. Every token of the GPT-2 vocabulary [25] is mapped to a codeword of 16 real channel symbols whose coordinates are organized in shells; every transmitted shell is scaled to unit power, so that any prefix is a valid transmission and an additional shell extends the prefix of exactly the tokens for which it is sent. The receiver decodes with the language model as its prior and requests further shells only for the tokens whose posterior confidence remains below a threshold. The system is also nested in time. The receiver's prior is shaped by three memories that evolve at three rates, namely the context of the conversation (every message, without signaling), a logit bias in embedding space (every ten adaptation steps) and low-rank adapter slots of the language model [26] (every fifty steps, one slot per knowledge domain). The multi-frequency organization follows the nested-learning view of a model as a set of components updated at different rates [27]; in a transceiver, however, each update frequency carries a price in channel uses, and DN-SC charges that price, bit by bit, in channel symbols.

The evaluation protocol was fixed in advance so that its outcome could refute the proposed design. Teacher-forced emulation, which scores each token given the true preceding tokens, is inexpensive but optimistic; the same received symbols are therefore also decoded free-running on a subset of the steps,

and a decision rule fixed in advance determines which measurement is reported; Section 5.3 declares one deviation from it. The decoder of the refinement policy is selected on validation text only, by a pre-specified rule, and the confirmatory comparison relies on six seeds, paired bootstrap intervals and an exact seed-level test.

The main contributions of this article are summarized as follows.

- A rate-nested token codebook with per-shell block power control is developed. Every transmitted block satisfies the power constraint with equality, which makes token-level incremental redundancy exact, and nesting causes no measurable loss at the prefix length that was examined (a channel-only token accuracy of 0.8885 against 0.8890 for a codebook trained for that length alone).

- A time-nested receiver with an exact synchronization ledger is formulated. The context memory is built by the receiver from its own decisions, whereas the logit-bias and adapter memories are learned at the transmitter and shipped as quantized differences at their own periods; adapter slots retain one adapter per domain, and every shipped bit is recorded.

- A complete cost model in channel symbols is derived, comprising the forward symbols, the return-link signaling of the refinement requests and the synchronization of the memories. It yields a net semantic rate, the traffic volume above which adaptation pays, and a separation benchmark that relates the prior cross-entropy to the number of symbols per token.

- A pre-specified evaluation shows that, on a hierarchical drift stream, DN-SC requires 23.3% fewer real channel symbols per token than a frozen fixed-rate receiver with the same decoder at 90% token accuracy (27.9% under free-running decoding), with the request signaling included and the synchronization accounted for separately. The analysis further shows that the context memory, which requires no synchronization, provides most of the reduction in prior cross-entropy achieved by the memories, and that shipped adaptation pays only above a large traffic volume.

The remainder of this article is organized as follows. Section 2 reviews related work. Section 3 presents the system model and formulates the problem. Section 4 details the DN-SC framework and its analytical properties. Section 5 describes the experimental setup, Section 6 reports and discusses the results, and Section 7 concludes the article.

2 | Related Work

This section reviews the two lines of research on which DN-SC builds. The first concerns learned and LLM-enabled semantic transmission of text; the second concerns rate adaptation at the receiver's request and the alignment of transmitter and receiver knowledge after deployment. The section closes with a positioning of the present work.

2.1 | LLM-Enabled Semantic Communication

Deep JSCC replaces separate source and channel codes by a single learned mapping from content to channel symbols. For text, Farsad et al. [10] encoded sentences with recurrent networks, and DeepSC [11] employed a Transformer [28] with a fixed number of symbols per word, trained and evaluated on

the Europarl corpus [29]; its restriction to sentences of 4–30 words has been adopted by many later studies. For images, deep JSCC avoids the cliff effect of separate coding [12] and admits bandwidth-agile, progressively refinable codes [13]. Surveys and tutorials place these designs within the broader program of semantic and goal-oriented communication [2, 7–9]. In this line of work the encoder and decoder are typically trained jointly for a data distribution that is assumed to be fixed, and the number of channel uses per message is either fixed or selected from a small set at the transmitter.

Pre-trained language models enter semantic communication in three roles. As a measure of importance, they rank words so that power or bandwidth can be allocated to the words that matter most [16]. As a knowledge base, large AI models supply the background knowledge of the transceiver and are regarded as a natural foundation for 6G semantic systems [17–19]; LLM-based text transceivers that dispense with a fixed knowledge base have been proposed [30], and large models have been fine-tuned and compressed for multi-task semantic communication [31]. As a prior, finally, the language model supplies the probability of the next token given the tokens already decoded, which the receiver combines with the channel likelihood. Wang et al. [14] built a text semantic communication system on this principle, with tokens as semantic units and a pre-trained LLM as the prior of the decoder, and Qiao et al. [15] exploited the cross-modal context among tokens through large models to save bandwidth. DN-SC belongs to this third group and adopts the LLM-prior decoder as its starting point.

2.2 | Rate Adaptation and Knowledge Alignment

Rate adaptation at the receiver's request has been studied as semantic hybrid automatic repeat request (HARQ). Jiang et al. [24] combined a learned sentence codec with retransmission and incremental knowledge, following the incremental-redundancy principle of classical HARQ [23]. Successive refinement [32] and nested representations [33] provide the structural basis of prefix codes whose leading part is useful on its own. DN-SC applies incremental redundancy per token rather than per message, conditions each request on the posterior confidence obtained with the language prior, and charges the requests on the return link.

A semantic receiver operates correctly only as long as its knowledge matches that of the transmitter. Zhang et al. [20] studied transceivers that must cope with a task-unaware transmitter and dynamic data; Zhang et al. [21] test whether the knowledge bases of the two ends match and adapt them by a lightweight step when they do not; and cooperative update policies keep the knowledge bases of several semantic communication pairs current while limiting the overhead of the update [22]. In machine learning, adaptation to a changing data stream raises the problem of catastrophic forgetting [34], low-rank adaptation (LoRA) [26] keeps the updates of a large model small, and communication-efficient distributed learning compresses updates by quantization [35, 36]. The nested-learning paradigm [27] organizes a model as components updated at different frequencies. In a transceiver, however, the receiver cannot compute its own updates without the source text, so that every update incurs a price in air time that grows with its size and frequency.

Table 1 positions DN-SC against the closest work along the dimensions studied in this article. DN-SC differs from the work above in that it charges every update of a language-model prior, at three timescales, bit by bit and in channel symbols, against the rate saved by that update, and thereby obtains the traffic volume above which adaptation pays. The table records the subject of each work; a dash does not imply

TABLE 1 | Positioning of DN-SC against representative work. ✓: addressed as a subject of the work; —: not addressed as a subject.

Work	Pre-trained language model in the link	Receiver-driven incremental redundancy	Knowledge or model adaptation after deployment	Update overhead as a design quantity	Several update timescales
Semantic HARQ [24]	—	✓	—	—	—
Importance-aware SC [16]	✓	—	—	—	—
LLM-enabled text SC [14]	✓	—	—	—	—
Token communications [15]	✓	—	—	—	—
Dynamic-data SC [20]	—	—	✓	—	—
Knowledge-base adaptation [21]	—	—	✓	—	—
Knowledge-base update [22]	—	—	✓	✓	—
DN-SC (this work)	✓	✓	✓	✓	✓

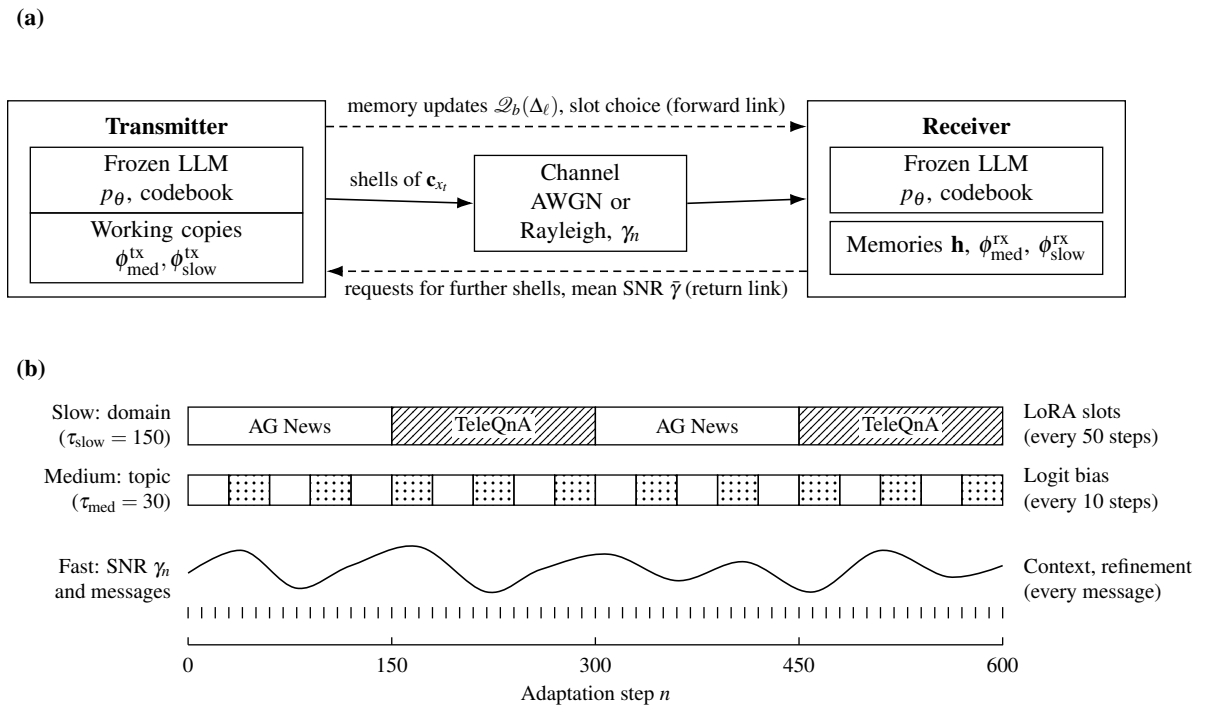


FIGURE 1 | System model. (a) Transmitter and receiver share a frozen language model and the codebook; the forward link carries the payload and the memory shipments, and the return link carries the requests for further shells. (b) The content drifts at three timescales, each tracked by one memory of the receiver; the stream used in the experiments is shown.

that the work could not be extended in that direction.

3 | System Model and Problem Formulation

This section specifies the link and the information available at each end, the model of the traffic and of its drift, the channel, and the performance metrics. It concludes with the optimization problem that the remainder of the article addresses. Figure 1 summarizes the model.

3.1 | Network, Traffic, and Drift Model

A transmitter, such as a base station or an edge server, sends text messages to a receiver over a wireless link. Both ends hold the same frozen pre-trained language model p_θ with vocabulary $\mathcal{V}$, $|\mathcal{V}| = V$, and

the same semantic codebook $\{\mathbf{c}_v\}_{v\in\mathscr{V}}$ with $\mathbf{c}_v \in \mathbb{R}^M$. A message is a token sequence $\mathbf{x} = (x_1, \ldots, x_T)$ with $x_t \in \mathscr{V}$, and token x_t is sent as a prefix of $\mathbf{c}_{x_t}$ of length $m \le M$ real channel symbols, that is, $m/2$ complex channel uses. The receiver additionally maintains three memories that shape its prior (Section 4.3). The transmitter knows the text, the codebook, the mean signal-to-noise ratio (SNR) fed back by the receiver and its own working copies of the memories, but neither the receiver's noise nor its decisions. The receiver knows its channel state, the number of tokens of each message (from the number of received symbol groups) and its own memories. A return link carries the receiver's requests for further symbols, and the forward link carries, in addition to the payload, the updates of the receiver's memories.

Traffic is organized in adaptation steps $n = 1, 2, \ldots$. At every step, each of N_c parallel conversations sends the next sentence of its current document, and a conversation that reaches the end of its document opens a new one. The content drifts at three timescales. The fast band comprises the channel and the document of each conversation. The medium band is the topic from which new documents are drawn; it changes every τ_{med} steps, and a new document is drawn from the current topic with probability κ and otherwise from a topic chosen uniformly at random. The slow band is the knowledge domain, which changes every τ_{slow} steps and returns after the other domains have been visited, so that the retention of earlier knowledge becomes observable. The SNR of step n, expressed in decibels, follows a clipped first-order autoregressive process,

$$\tilde{\gamma}_n = \mu_\gamma + \rho\,(\tilde{\gamma}_{n-1} - \mu_\gamma) + \sigma_\gamma\sqrt{1-\rho^2}\,\varepsilon_n, \qquad \gamma_n = \min\{\gamma_{\max}, \max\{\gamma_{\min}, \tilde{\gamma}_n\}\}, \tag{1}$$

with independent $\varepsilon_n \sim \mathcal{N}(0,1)$ and $\tilde{\gamma}_0 = \mu_\gamma$, so that the stationary standard deviation of $\tilde{\gamma}_n$ equals σ_γ.

3.2 | Channel Model

Each transmitted block, defined as one shell of the codewords of the tokens of one message that a transmission round carries (Section 4.1), is scaled to unit average power per symbol by a gain. After coherent equalization with perfect channel state information, symbol j of token t is received as

$$y_{t,j} = c_{x_t,j} + w_{t,j}, \qquad w_{t,j} \sim \mathcal{N}\left(0, \lambda_{t,j}^{-1}\right), \qquad \lambda_{t,j} = g_{t,k(j)}^2\,|h|^2\,10^{\gamma_n/10}, \tag{2}$$

where $k(j)$ denotes the shell containing coordinate j, $g_{t,k}$ the gain $g_k(\mathscr{T})$ of (10) of the block that carried shell k of token t, $|h|^2 = 1$ for the additive white Gaussian noise (AWGN) channel and $|h|^2 \sim \text{Exp}(1)$, drawn once per message and floored at $|h| = 0.05$, under Rayleigh block fading. The noise is independent across symbols, tokens and messages. Signaling and synchronization bits are charged at the capacity of $\bar{\gamma} = 10^{\mu_\gamma/10}$, the linear value of the mean SNR in decibels,

$$C(\bar{\gamma}) = \tfrac{1}{2}\log_2(1+\bar{\gamma}) \quad \text{bits per real channel symbol}, \tag{3}$$

which equals 1.7297 at 10 dB. This charge is optimistic and applies equally to every scheme that signals or synchronizes.

3.3 | Performance Metrics and Problem Formulation

Fidelity is measured by the token accuracy, and the knowledge of the receiver by the cross-entropy of its prior on the transmitted tokens,

$$A = \frac{1}{N_{\text{tok}}} \sum_{t=1}^{N_{\text{tok}}} \mathbb{1}\{\hat{x}_t = x_t\}, \qquad \bar{H} = -\frac{1}{N_{\text{tok}}} \sum_{t=1}^{N_{\text{tok}}} \log_2 \pi_t(x_t) \quad \text{bits per token}, \tag{4}$$

where $\hat{x}_t$ is the receiver's decision and π_t its prior for position t (Section 4.2); BLEU-4 [37] is reported where complete sentences are compared. Cost is measured in real channel symbols per token and comprises three parts: the forward symbols S_{fwd} of the payload, the return-link signaling B_{fb} of the refinement requests in bits per token, and the synchronization $\bar{B}_{\text{sync}}$ of the receiver's memories in bits per adaptation step; B_{fb} is either of the two signaling costs defined in Section 4.4. If one adaptation step serves N messages of $\bar{T}$ tokens on average, a receiver policy ϖ, consisting of its memories, decoder and refinement rule, incurs the total cost

$$S_{\text{tot}}(\varpi; N) = S_{\text{fwd}}(\varpi) + \frac{B_{\text{fb}}(\varpi)}{C(\bar{\gamma})} + \frac{\bar{B}_{\text{sync}}(\varpi)}{C(\bar{\gamma})\,\bar{T}\,N} \tag{5}$$

real channel symbols per token. The design problem is to minimize this cost subject to a target token accuracy $A^\star$,

$$\varpi^\star(N) = \arg\min_{\varpi} S_{\text{tot}}(\varpi; N) \qquad \text{subject to} \qquad A(\varpi) \geq A^\star. \tag{6}$$

The first two terms of (5) are measured per message, whereas the third depends on the number of messages served by one update, which is a property of the deployment rather than of the codec. The experiments therefore report $S_{\text{fwd}} + B_{\text{fb}}/C(\bar{\gamma})$ at $A^\star = 0.9$ as the primary endpoint and treat the synchronization term as a function of N (Section 4.5).

4 | Proposed DN-SC Framework

This section develops the DN-SC transceiver. It first describes the rate-nested codebook and its power control, then the Bayesian receiver with its language prior, the time-nested memories and their synchronization, and the token-level refinement protocol. It closes with the cost model and the analytical properties on which the evaluation relies. Figure 2 shows the architecture of the transceiver.

4.1 | Rate-Nested Codebook and Block Power Control

Let $\mathbf{e}_v \in \mathbb{R}^D$ denote the input embedding of token v in the frozen language model and s_e the standard deviation of all entries of the embedding table. A small network maps the scaled embedding $\bar{\mathbf{e}}_v = \mathbf{e}_v / s_e$ to a raw codeword,

$$\tilde{\mathbf{c}}_v = \mathbf{W}_1 \bar{\mathbf{e}}_v + \mathbf{W}_3\, \text{GELU}(\mathbf{W}_2 \bar{\mathbf{e}}_v + \mathbf{b}_2), \tag{7}$$

with $\mathbf{W}_1 \in \mathbb{R}^{M \times D}$, $\mathbf{W}_2 \in \mathbb{R}^{D_{\text{h}} \times D}$, $\mathbf{b}_2 \in \mathbb{R}^{D_{\text{h}}}$ and $\mathbf{W}_3 \in \mathbb{R}^{M \times D_{\text{h}}}$. Since $\mathbf{W}_3$ is initialized at zero, training starts from a linear projection of the embeddings, and no language model has to run at the transmitter to encode. The M coordinates are partitioned into K shells at the prefix lengths $m_1 < m_2 < \cdots < m_K = M$; shell k holds the coordinates $\mathcal{S}_k = \{m_{k-1}+1, \ldots, m_k\}$, with $m_0 = 0$, and has width $d_k = m_k - m_{k-1}$. Each

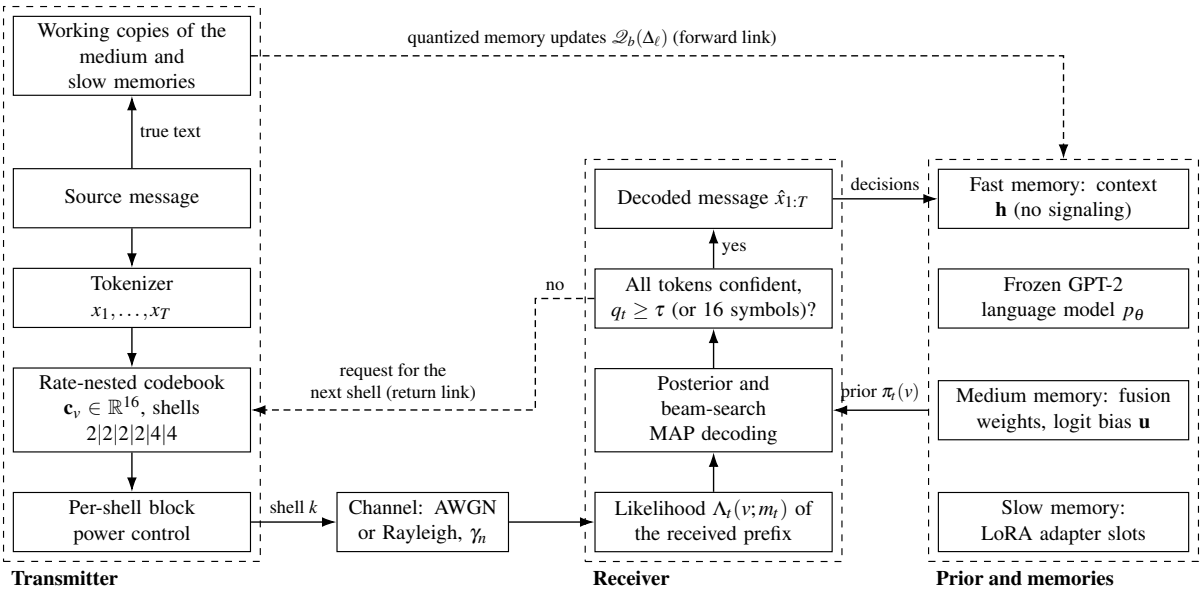


FIGURE 2 | Architecture of DN-SC. Solid arrows denote the forward data path; dashed arrows denote the return-link requests for further shells and the shipments that synchronize the medium and slow memories of the receiver, both of which are charged in channel symbols. The fast memory (context) is built by the receiver from its own decisions and requires no signaling.

shell is centered and scaled under the unigram distribution p_{u} of the base corpus,

$$\hat{\mathbf{c}}_{v,k} = \frac{\tilde{\mathbf{c}}_{v,\mathscr{S}_k} - \mu_k}{\sqrt{P_k}}, \qquad \mu_k = \sum_{v \in \mathscr{V}} p_{\mathrm{u}}(v)\, \tilde{\mathbf{c}}_{v,\mathscr{S}_k}, \qquad P_k = \frac{1}{d_k} \sum_{v \in \mathscr{V}} p_{\mathrm{u}}(v) \left\| \tilde{\mathbf{c}}_{v,\mathscr{S}_k} - \mu_k \right\|^2, \tag{8}$$

and the energy of every codeword in the shell is limited to ζ times the shell average,

$$\mathbf{c}_{v,\mathscr{S}_k} = \min\left\{ 1, \frac{\sqrt{\zeta\, d_k}}{\|\hat{\mathbf{c}}_{v,k}\|} \right\} \hat{\mathbf{c}}_{v,k}. \tag{9}$$

Hence every shell has zero mean before the peak limit and an average power of at most one per symbol under p_{u}.

This constraint holds on average over the base corpus, not for a particular message. Before transmission, shell k of the set $\mathscr{T}$ of tokens of a message that a round carries, namely all T tokens in the first round and the requested tokens in a refinement round, is multiplied by

$$g_k(\mathscr{T}) = \left(\frac{|\mathscr{T}|\, d_k}{\sum_{t \in \mathscr{T}} \|\mathbf{c}_{x_t,\mathscr{S}_k}\|^2} \right)^{1/2}, \tag{10}$$

so that every transmitted block has an average power of exactly one per symbol. The gain is applied to the whole block, pilots included, and is absorbed into the channel estimate; the receiver observes it only through the precision $\lambda_{t,j}$ in (2). Consequently, no scheme benefits from messages whose codewords have above-average energy.

Proposition 1 (exact incremental refinement). Every transmitted block satisfies the power constraint with equality, and hence so does every prefix. If the channel state ($|h|$ and γ_n) is constant over the rounds of a message, receiving shells $1, \ldots, k$ of all tokens in k separate rounds yields observations with the same

distribution as receiving the prefix of length m_k at once, and a refinement round extends the prefix of each requested token by one shell, received with the precision determined by the gain of the requested set.

Proof. By (10), the gain of a block depends only on the coordinates of its own shell and on the tokens it carries, and by (2) the noise is independent across symbols. Hence neither a block nor its noise depends on when, or whether, other shells are transmitted, and with a constant channel state the precision of each symbol is the same as in a single transmission. Since each block has unit average power per symbol, the average over the blocks of any prefix is also one. □

The codebook is trained once, before deployment and without the language model, by maximizing the channel-only posterior of the transmitted token at randomly drawn prefix lengths and SNRs,

$$\mathscr{L}_{\mathrm{cb}}(\mathbf{W}) = \mathbb{E}_{v\sim\bar{p}_{\mathrm{u}},\, \gamma\sim\mathscr{U}[\gamma_{\mathrm{lo}},\gamma_{\mathrm{hi}}],\, \mathbf{w}} \left[\frac{1}{|\mathscr{M}_b|} \sum_{m\in\mathscr{M}_b} -\log \frac{\exp\Lambda(v;m)}{\sum_{v'\in\mathscr{V}} \exp\Lambda(v';m)} \right], \tag{11}$$

where $\Lambda(v;m)$ is the log-likelihood (12) below for a single token with $\lambda_{t,j} = 10^{\gamma/10}$, γ_{lo} and γ_{hi} bound the training SNR in decibels, $\mathscr{M}_b$ is a set of prefix lengths drawn afresh for every batch, and $\bar{p}_{\mathrm{u}}$ mixes the unigram distribution of the base corpus with the uniform distribution over $\mathscr{V}$, so that tokens absent from the base corpus, which form the vocabulary of later domains, are also separated. Because all prefixes are trained jointly, each prefix constitutes a usable code.

4.2 | Bayesian Receiver With a Language Prior

When the first m symbols of token t have been received, the Gaussian log-likelihood of every vocabulary entry follows from (2), up to a term that does not depend on v, as

$$\Lambda_t(v;m) = \sum_{j=1}^{m} \lambda_{t,j} \left(y_{t,j}\, c_{v,j} - \tfrac{1}{2}\, c_{v,j}^2 \right), \tag{12}$$

which is evaluated for all V entries by one matrix product of the received symbols with the codebook. The prior is the frozen language model, modified by the memories of the receiver. The slow memory consists of K_{s} LoRA adapters; adapter s replaces each adapted weight matrix $\mathbf{W}^{(i)}$ of the language model by

$$\mathbf{W}_s^{(i)} = \mathbf{W}^{(i)} + \frac{\alpha}{r}\, \mathbf{B}_s^{(i)} \mathbf{A}_s^{(i)}, \qquad \mathbf{A}_s^{(i)} \in \mathbb{R}^{r\times D_{\mathrm{in}}^{(i)}}, \quad \mathbf{B}_s^{(i)} \in \mathbb{R}^{D_{\mathrm{out}}^{(i)}\times r}, \tag{13}$$

where $D_{\mathrm{in}}^{(i)}$ and $D_{\mathrm{out}}^{(i)}$ are the input and output widths of the matrix, r is the rank and α the scaling [26], and $\phi_s = \{\mathbf{A}_s^{(i)}, \mathbf{B}_s^{(i)}\}_i$ denotes the parameters of slot s ($\phi_\varnothing$: no adapter). With $\hat{\mathbf{x}}_{<t}$ the tokens already decided in the message and $\mathbf{h}$ the context, the prior of the receiver is

$$\log \pi_t(v) = \log \mathrm{softmax}_v \left[w_{\mathrm{p}} \log p_\theta(v \mid \mathbf{h}, \hat{\mathbf{x}}_{<t}; \phi_s) + \frac{1}{s_e}\, \mathbf{e}_v^\top \mathbf{u} \right], \tag{14}$$

where the weight w_{p} sharpens or flattens the language model and $\mathbf{u} \in \mathbb{R}^D$ defines a logit bias in embedding space, which shifts the probability of all tokens that are close in meaning by means of a single vector. The posterior of token t after m_t symbols, the decision of a greedy decoder and the receiver's confidence

in the decision are

$$p_t(v) = \frac{\pi_t(v)\, e^{w_\mathrm{l}\Lambda_t(v;m_t)}}{Z_t}, \qquad Z_t = \sum_{v' \in \mathscr{V}} \pi_t(v')\, e^{w_\mathrm{l}\Lambda_t(v';m_t)}, \qquad \hat{x}_t = \arg\max_v p_t(v), \qquad q_t = p_t(\hat{x}_t), \quad (15)$$

with a likelihood weight w_l; before any adaptation, $w_\mathrm{p} = w_\mathrm{l} = 1$ and $\mathbf{u} = \mathbf{0}$. Under beam search, $\hat{x}_t$ is instead the token at position t of the selected sentence (16), and $q_t = p_t(\hat{x}_t)$ is evaluated along that sentence.

Decoding is free-running, that is, the language model is conditioned on the receiver's own earlier decisions, so that an early error misleads the prior for the remainder of the sentence. The receiver approximates instead the sequence maximum a posteriori (MAP) decision

$$\hat{\mathbf{x}} = \arg\max_{\mathbf{x} \in \mathscr{V}^T} J(\mathbf{x}), \qquad J(x_{1:t}) = \sum_{i=1}^{t} \Big[\log \pi_i(x_i \mid x_{<i}) + w_\mathrm{l} \Lambda_i(x_i; m_i) \Big], \quad (16)$$

in which J equals the log-posterior of the partial sentence up to a constant (a tempered log-posterior when $w_\mathrm{l} \neq 1$), by a beam search of width B that retains, at every position, the B best extensions of the surviving partial sentences,

$$\mathscr{B}_t = \text{top-}B_{(x_{1:t-1},v)\,:\, x_{1:t-1} \in \mathscr{B}_{t-1},\, v \in \mathscr{V}}\; J(x_{1:t-1}, v), \qquad \mathscr{B}_0 = \{\varnothing\}. \quad (17)$$

The ranking must use the joint score J. Summing the per-position posteriors of (15) instead gives

$$\sum_{i=1}^{t} \log p_i(x_i) = J(x_{1:t}) - \sum_{i=1}^{t} \log Z_i(x_{<i}), \quad (18)$$

where $Z_i(x_{<i})$ is, for $w_\mathrm{l} = 1$, proportional to the predictive likelihood of the observation of position i given the prefix. Such a ranking penalizes exactly those prefixes under which the next observation is well explained, which is a form of the label-bias problem [38]. The confidence q_t reported for a decision is that of the posterior at position t along the selected path.

4.3 | Time-Nested Memories and Their Synchronization

The prior (14) is shaped by three memories that change at three rates (Figure 3b). The fast memory is the context: a message that continues a document is decoded with the receiver's own reconstruction of the preceding messages of the conversation as $\mathbf{h}$, up to a fixed number of tokens. This is the native memory of the language model; it changes with every message, requires no gradient and no air time, but its errors propagate, since a wrongly decoded message becomes a wrong context.

The medium memory $\phi_\mathrm{med} = (w_\mathrm{p}, w_\mathrm{l}, \mathbf{u})$ and the slow memory $\phi_\mathrm{slow} = \{\phi_1, \ldots, \phi_{K_\mathrm{s}}\}$ can only be learned with the true text. They are learned at the transmitter, which maintains a working copy ϕ^{tx} of both and, at every adaptation step n, evaluates the posterior cross-entropy of the N_n tokens of the step's messages with the true earlier tokens and the true context $\mathbf{h}_i^{\mathrm{tx}}$,

$$\mathscr{L}_n(\phi) = \frac{1}{|\mathscr{M}_n|} \sum_{m \in \mathscr{M}_n} \frac{1}{N_n} \sum_{(i,t)} -\log p_t^{\phi}\left(x_{i,t} \mid \mathbf{y}_{i,t,1:m}, \mathbf{x}_{i,<t}, \mathbf{h}_i^{\mathrm{tx}}\right), \quad (19)$$

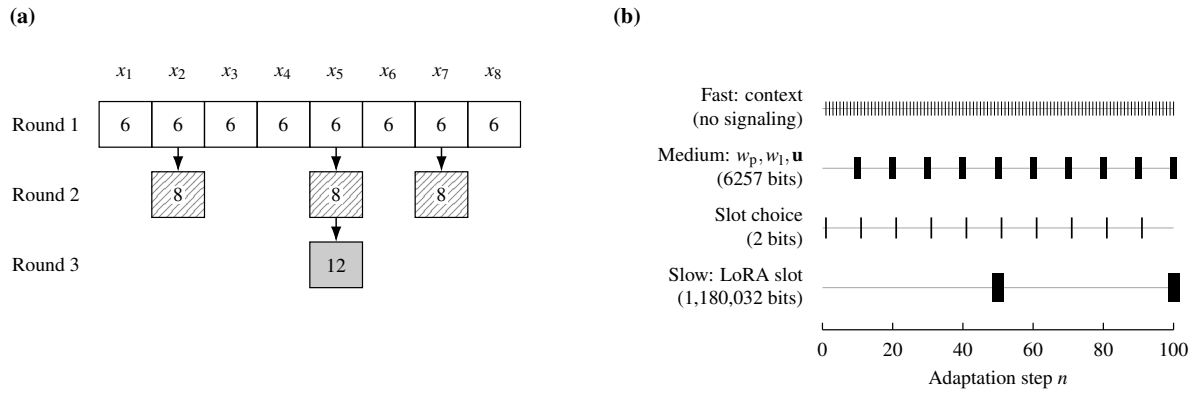


FIGURE 3 | (a) Selective refinement of one message of eight tokens with a first round of 6 symbols per token (illustrative). Each cell gives the number of symbols received for the token after the round, and each arrow a request for the next shell on the return link; the mean cost is $(5\cdot 6+2\cdot 8+12)/8 = 7.25$ symbols per token in three rounds. (b) Instants at which each memory reaches the receiver during the first 100 adaptation steps, with the cost of one shipment at $b = 8$ bits per parameter; averaged over the stream, the synchronization amounts to 24.23 kbit per adaptation step (bit ledger of the experiments).

averaged over the operating prefix length and one further prefix length drawn at random ($\mathscr{M}_n$), so that the prior serves every rate. Each level $\ell \in \{\text{med}, \text{slow}\}$ takes one optimizer step per adaptation step with its own learning rate η_ℓ and the gradient norm clipped at 1.0,

$$\phi^{\text{tx}}_{\ell,n} = \phi^{\text{tx}}_{\ell,n-1} - \eta_\ell \,\text{Adam}\big(\nabla_{\phi_\ell} \mathscr{L}_n\big)\,, \tag{20}$$

and within the slow memory only the active slot is updated. Both ends initialize every slot identically (**A** from a shared random seed, $\mathbf{B} = \mathbf{0}$), so that only differences need to be shipped. The receiver holds its own copy ϕ^{rx}_ℓ, which changes only at the period C_ℓ of the level. When $n \equiv 0 \pmod{C_\ell}$, the transmitter quantizes the difference $\blacksquare = \phi^{\text{tx}}_\ell - \phi^{\text{rx}}_\ell$ of every parameter tensor with b bits per entry, and both ends apply the quantized difference:

$$\mathscr{Q}_b(\blacksquare) = \frac{a}{L}\,\text{round}\left(\frac{L\blacksquare}{a}\right), \qquad a = \|\blacksquare\|_\infty, \qquad L = 2^{b-1}-1, \qquad \phi^{\text{rx}}_\ell \leftarrow \phi^{\text{rx}}_\ell + \mathscr{Q}_b(\blacksquare), \qquad \phi^{\text{tx}}_\ell \leftarrow \phi^{\text{rx}}_\ell. \tag{21}$$

Shipments take place after the messages of a step have been decoded. Between two shipments the receiver therefore decodes with a stale copy: if $\phi^{\text{rx}}_\ell(n')$ denotes its copy at the end of step n', the messages of step n are decoded with

$$\phi^{\text{rx}}_\ell(C_\ell \lfloor (n-1)/C_\ell \rfloor)\,, \tag{22}$$

so that the period C_ℓ trades staleness against air time; all receivers compared in this article learn in the same way and differ only in when each level reaches the receiver. A shipment of level ℓ costs $B_\ell = \sum_i c_i$ bits, with $c_i = 32 + b\,n_i$ if the difference of tensor i is nonzero and $c_i = 1$ (a flag bit) otherwise, where n_i is the number of entries of the tensor and 32 bits carry its scale a; for the slow memory, the sum runs over the tensors of the slot being shipped. The slot choice described below costs $\lceil \log_2(K_\text{s}+1) \rceil$ bits every C_sel steps, and the average synchronization rate is

$$\bar{B}_\text{sync} = \frac{B_\text{med}}{C_\text{med}} + \frac{B_\text{slow}}{C_\text{slow}} + \frac{\lceil \log_2(K_\text{s}+1) \rceil}{C_\text{sel}} \quad \text{bits per adaptation step.} \tag{23}$$

Algorithm 1 One adaptation step n of DN-SC

Input: messages $\mathbf{x}_1, \ldots, \mathbf{x}_{N_c}$ of the step; SNR γ_n; first-round prefix m_{k_0} and threshold τ (Section 4.4); receiver copies ϕ^{rx} and contexts $\mathbf{h}_i^{\mathrm{rx}}$; transmitter copies ϕ^{tx} and contexts $\mathbf{h}_i^{\mathrm{tx}}$
Output: decoded messages $\hat{\mathbf{x}}_i$; updated memories; bit ledger

if $(n-1) \bmod C_{\mathrm{sel}} = 0$ **then** ▷ slot choice, Eq. (24)
  evaluate H_s under ϕ^{rx}, choose s_{new} and signal it; ledger $+= \lceil \log_2(K_{\mathrm{s}}+1) \rceil$
end if
for each message i **do** ▷ transmission and decoding
  encode $\mathbf{x}_i$ by Eqs. (7)–(10) and send round 1, m_{k_0} symbols per token
  repeat
    decode by beam search, Eqs. (12)–(17), with prior ϕ^{rx} and context $\mathbf{h}_i^{\mathrm{rx}}$
    close every open token with $q_t \geq \tau$; request the next shell for the others ▷ Eqs. (25), (26)
    send the next shell of the requested tokens at unit power, Eq. (10)
  until no token is open
  $\mathbf{h}_i^{\mathrm{rx}} \leftarrow \mathbf{h}_i^{\mathrm{rx}} \| \hat{\mathbf{x}}_i$; $\mathbf{h}_i^{\mathrm{tx}} \leftarrow \mathbf{h}_i^{\mathrm{tx}} \| \mathbf{x}_i$ ▷ fast memory
end for
update $\phi_{\mathrm{med}}^{\mathrm{tx}}$ and the active slot of $\phi_{\mathrm{slow}}^{\mathrm{tx}}$ by Eqs. (19) and (20)
for $\ell \in \{\mathrm{med}, \mathrm{slow}\}$ with $n \equiv 0 \pmod{C_\ell}$ **do**
  ship $\mathcal{Q}_b(\phi_\ell^{\mathrm{tx}} - \phi_\ell^{\mathrm{rx}})$ and set both copies equal; ledger $+= B_\ell$ ▷ Eq. (21)
end for

The multi-frequency organization follows the nested-learning view [27], and the quantized, bit-counted synchronization follows the practice of communication-efficient learning [35, 36].

Proposition 2 (synchronization). Immediately after a shipment of level ℓ, the two copies are bit-identical, and every entry of the applied difference deviates from the unquantized difference by at most $a/(2L)$. If only the active slot of the slow memory changes between two shipments and the cost B_ℓ of every shipment of a level is the same, the average number of synchronized bits per adaptation step over any horizon that is a multiple of C_{med}, C_{slow} and C_{sel} equals (23).

Proof. Both ends add the same quantized difference to the same copy, and the transmitter then resets its working copy to the result, which proves the first claim. Rounding to the grid $\{ja/L\}$ moves every entry by at most half a grid step, which proves the second. In each period of length C_ℓ exactly one shipment of cost B_ℓ occurs, because only one tensor set per level has changed, and the slot choice is signaled once every C_{sel} steps; averaging over a horizon that contains a whole number of each period gives (23). □

Adapting a single adapter to one domain can damage the prior for another, so the slow memory keeps one adapter per domain. Every C_{sel} steps, the transmitter evaluates, on the messages it is about to send, the prior cross-entropy H_s in bits per token of the receiver's copy with every choice $s \in \{\varnothing, 1, \ldots, K_{\mathrm{s}}\}$, and changes the active slot s_{cur} only if another choice is better by a margin δ:

$$s^\star = \arg\min_s H_s, \qquad s_{\mathrm{new}} = \begin{cases} s^\star, & H_{s^\star} < H_{s_{\mathrm{cur}}} - \delta, \\ s_{\mathrm{cur}}, & \text{otherwise.} \end{cases} \tag{24}$$

Whenever $s_{\mathrm{new}} = \varnothing$ and an untrained slot remains, that slot is started instead. A returning domain thus finds its adapter intact; recalling it costs only the two-bit slot signal, after which its updates are shipped as usual, and the receiver keeps an adapter that is worse than none by more than δ for at most C_{sel} steps. Algorithm 1 states one adaptation step, and Figure 4 depicts it as a flowchart.

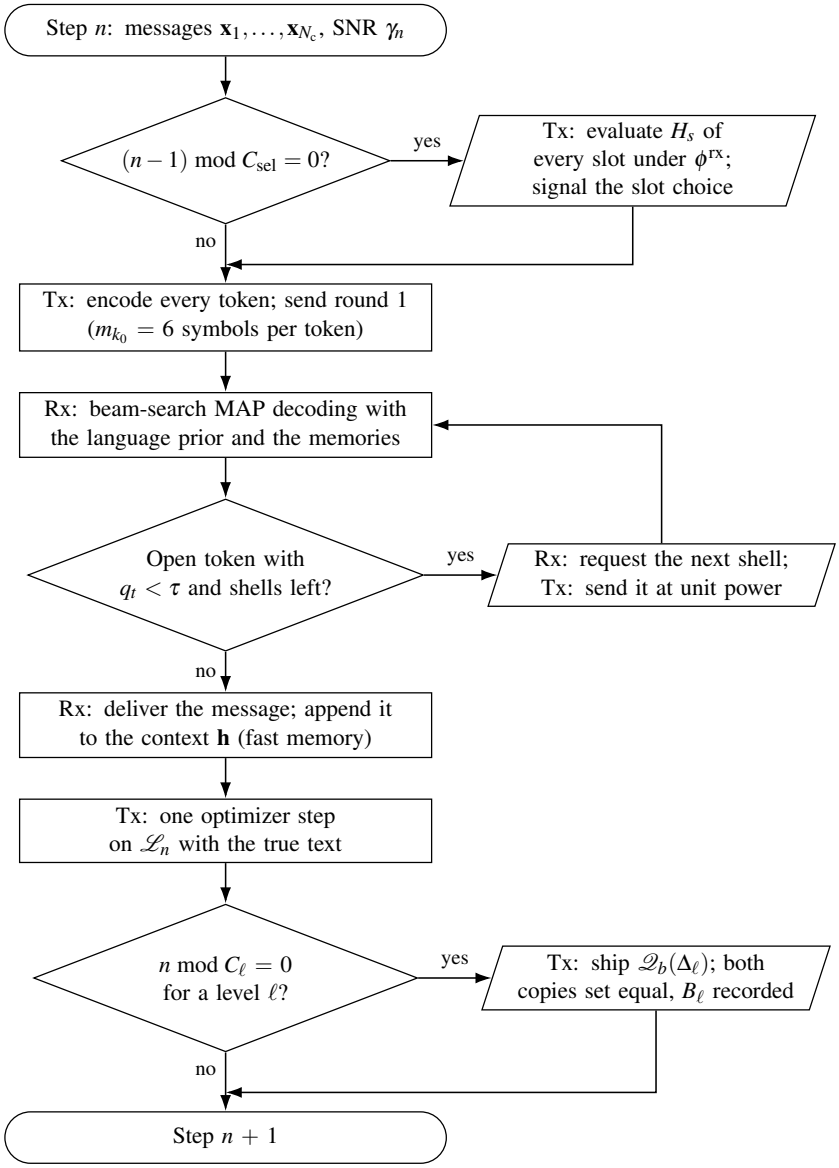


FIGURE 4 | Flowchart of one adaptation step of DN-SC (Algorithm 1). Tx and Rx indicate the end at which an operation is performed; parallelograms denote transmissions of signaling or synchronization data.

4.4 | Token-Level Selective Refinement

Round 1 delivers the first m_{k_0} symbols of every token. After each round, the receiver decodes the whole message again with the symbols received so far and decides, for every token that is still open, whether to request its next shell. A token that is not continued is closed permanently, and no decision is required after the last shell (Figure 3a). Under this rule, token t stops at the shell

$$k_t(\tau) = \min\left\{k \in \{k_0,\ldots,K\} : q_t^{(k)} \geq \tau\right\} \quad \text{(or } K \text{ if no such } k \text{ exists)}, \qquad S_{\text{fwd}}(\tau) = \frac{1}{N_{\text{tok}}}\sum_t m_{k_t(\tau)}, \tag{25}$$

where $q_t^{(k)}$ is the confidence of token t when the message is decoded with m_k symbols for that token, and S_{fwd} is the mean number of forward symbols per token over the N_{tok} tokens of an adaptation step; operating curves average these step means over the steps. By Proposition 1, the additional symbols extend what has already been received, which constitutes incremental redundancy at the level of individual

tokens, spent on the tokens that the language prior cannot resolve.

Let n_ν denote the number of tokens that are open after round ν and must be decided upon, and $c_\nu \le n_\nu$ the number of them that are continued. Two costs of the requests are derived from the decisions actually taken, in bits per token, namely an explicit bitmap and the empirical entropy of the decisions of each round, which an ideal entropy coder of the bitmap would approach:

$$B_{\mathrm{bm}} = \frac{1}{N_{\mathrm{tok}}} \sum_\nu n_\nu, \qquad B_{\mathrm{H}} = \frac{1}{N_{\mathrm{tok}}} \sum_\nu n_\nu \, h_2\!\left(\frac{c_\nu}{n_\nu}\right), \qquad h_2(p) = -p\log_2 p - (1-p)\log_2(1-p). \quad (26)$$

Both are converted into real channel symbols at $C(\bar{\gamma})$ and added to the forward symbols. The entropy B_{H} is a lower reference rather than an implemented code, whereas the bitmap is directly implementable; a fixed-prefix receiver requires no signaling at all.

Proposition 3 (signaling bounds). For the protocol without reopening, in which a closed token is never requested again, and for every threshold τ, $0 \le B_{\mathrm{H}} \le B_{\mathrm{bm}} \le K - k_0$ bits per token.

Proof. Since $0 \le h_2(p) \le 1$, $B_{\mathrm{H}} \le B_{\mathrm{bm}}$. A token takes part in one decision after each round in which it is open and has a shell left, which happens at most $K - k_0$ times; summing over tokens gives $\sum_\nu n_\nu \le (K-k_0)N_{\mathrm{tok}}$. □

4.5 | Cost Model and Analytical Properties

Each receiver is characterized by its measured operating curve $\{(S_i, A_i)\}$, the cost against the accuracy over the thresholds τ or the prefix lengths, sorted by cost. The cost at the target accuracy is read by linear interpolation,

$$S(A^\star) = S_i + \frac{A^\star - A_i}{A_{i+1} - A_i}\left(S_{i+1} - S_i\right) \quad \text{for } A_i < A^\star \le A_{i+1}, \quad (27)$$

and, when even the cheapest point of a curve exceeds the target ($A_1 \ge A^\star$), at that point, $S(A^\star) = S_1$, which is then an upper bound of the cost at the target. Let $R = \bar{T}\,[S_{\mathrm{fwd}} + B_{\mathrm{fb}}/C(\bar{\gamma})]$ be the resulting payload of a receiver per message. If one adaptation step, with $\bar{B}_{\mathrm{sync}}$ synchronized bits, serves N messages, the net semantic rate and the traffic above which the receiver is cheaper than a reference receiver with payload $R^{\mathrm{ref}} > R$ and no synchronization are

$$R_{\mathrm{net}}(N) = R + \frac{\bar{B}_{\mathrm{sync}}}{C(\bar{\gamma})N} \quad \text{real symbols per message}, \quad (28)$$

$$N^\star = \frac{\bar{B}_{\mathrm{sync}}}{C(\bar{\gamma})\,(R^{\mathrm{ref}} - R)}. \quad (29)$$

Equation (28) is (5) multiplied by $\bar{T}$.

Proposition 4 (crossover). $R_{\mathrm{net}}(N)$ decreases strictly in N, $R_{\mathrm{net}}(N) < R^{\mathrm{ref}}$ if and only if $N > N^\star$, and the relative change $R_{\mathrm{net}}(N)/R^{\mathrm{ref}} - 1$ tends to $R/R^{\mathrm{ref}} - 1$ as $N \to \infty$.

Proof. The second term of (28) is positive and inversely proportional to N. The inequality $R + \bar{B}_{\mathrm{sync}}/(C(\bar{\gamma})N) < R^{\mathrm{ref}}$ rearranges to $N > N^\star$, and the limit follows because the second term vanishes. □

The prior cross-entropy (4) also determines the cost of an idealized separate design that compresses each token with an arithmetic code matched to the receiver's prior and transmits the compressed bits with

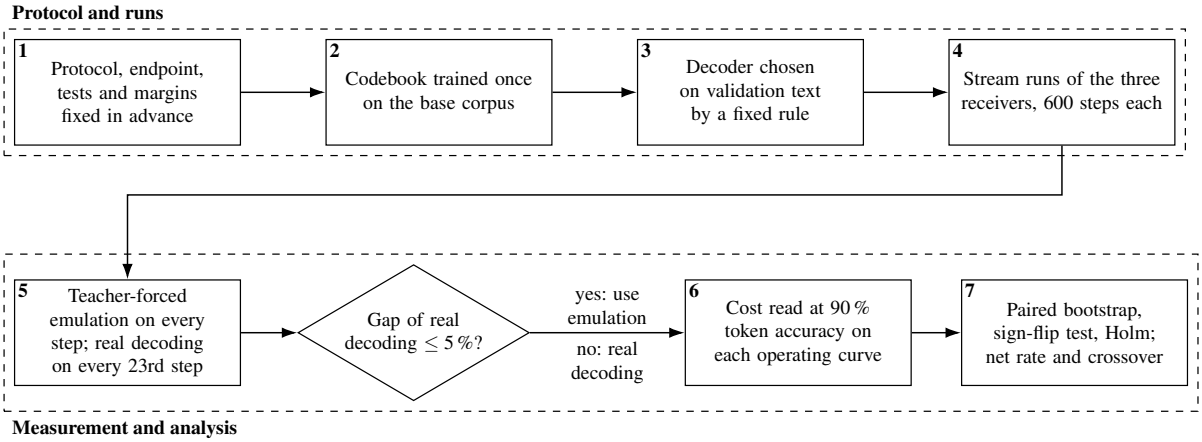


FIGURE 5 | Evaluation procedure. Steps 1 to 3 were completed before the confirmatory stream runs. The decision in the second row is the pre-specified rule that selects the measurement on which the reported costs rest; Section 5.3 states how it was applied.

a capacity-achieving channel code. Such a design requires on average

$$S_{\mathrm{sep}} = \frac{\bar{H}}{C(\bar{\gamma})} \quad \text{real symbols per token} \tag{30}$$

for lossless delivery in the limit of long blocks [5], so that a reduction $\Delta\bar{H}$ of the prior cross-entropy lowers this benchmark by $\Delta\bar{H}/C(\bar{\gamma})$. The benchmark ignores the penalty of finite block lengths [39] and the cliff effect at an unknown SNR, and it is used here to relate the knowledge of the receiver to its cost in channel symbols.

The receiver evaluates (12) for all V entries at a cost of $O(Vm)$ operations per token and runs the language model once per token and beam in every decoding pass; selective refinement repeats the pass for the messages that still contain open tokens, so that the number of passes per message equals the number of rounds. The transmitter performs one forward and one backward pass of the language model per adaptation step, the backward pass only through the adapted layers, and one forward pass per candidate slot every C_{sel} steps. The codebook is trained once and never shipped. Section 6.5 reports the measured parameter counts, latencies and run times.

5 | Experimental Setup

This section describes the data, the model and its parameters, the receivers that are compared, the selection of the decoder, and the protocol under which the results were obtained. The protocol, the primary endpoint, the tests and the equivalence margins were fixed in the project repository before the confirmatory runs. Figure 5 outlines the procedure.

5.1 | Data, Model, and Parameters

The frozen prior is GPT-2 [25], with 124,439,808 parameters, a vocabulary of $V = 50{,}257$ tokens and an embedding width of $D = 768$. The codebook uses $M = 16$ real symbols per token, nested at the prefix lengths $\{2,4,6,8,12,16\}$ (shell widths 2|2|2|2|4|4), with $D_{\mathrm{h}} = 256$ hidden units and a peak ratio of $\zeta = 4$, which amounts to 213,248 parameters. It was trained once on the base corpus, the Europarl

TABLE 2 | Simulation parameters.

Group	Parameter	Value
Language prior	Model; parameters	GPT-2 (frozen); 124,439,808
	Vocabulary V; embedding width D	50,257; 768
Codebook	Symbols per token M; prefix lengths m_k	16; 2, 4, 6, 8, 12, 16
	Hidden units D_{h}; peak ratio ζ; parameters	256; 4; 213,248
	Training steps; tokens per batch; uniform share	2000; 256; 25%
	Training SNR	$\mathscr{U}[0,20]$ dB per token
Stream	Domains, in order	AG News, TeleQnA, AG News, TeleQnA
	τ_{slow}; τ_{med}; κ; N_{c}; steps	150; 30; 0.9; 6; 600
Channel	Type; μ_γ; ρ; σ_γ; range	AWGN; 10 dB; 0.95; 4 dB; 0–20 dB
Fast memory	Context length	48 tokens
Medium memory	Content; learning rate; period C_{med}	w_{p}, w_{l}, $\mathbf{u} \in \mathbb{R}^{768}$; 10^{-3}; 10 steps
Slow memory	Slots K_{s}; LoRA rank r; α; layers	2; 8; 16; last 6 of 12
	Parameters per slot; learning rate; period C_{slow}	147,456; 5×10^{-4}; 50 steps
Slot choice	Period C_{sel}; margin δ; cost	10 steps; 0.05 bits/token; 2 bits
Synchronization	Bits per entry b; scale per tensor; per shipment	8; 32 bits; 6257 (medium), 1,180,032 (slot)
Decoder	Beam width B; first round m_{k_0}; reopening	4; 6 symbols; no
Refinement	Thresholds τ	0.5, 0.7, 0.9, 0.95, 0.99
Endpoint	Target token accuracy $A^\star$	0.90

sample distributed with NLTK [29] (14,006 sentences of 4–30 words, split 80/10/10 by document), for 2000 steps of 256 tokens, of which a quarter were drawn uniformly from the vocabulary, with one SNR per token drawn uniformly in $[\gamma_{\mathrm{lo}}, \gamma_{\mathrm{hi}}] = [0,20]$ dB and three prefix lengths per batch, using AdamW with a learning rate of 2×10^{-3}, 100 warm-up steps, cosine decay and gradient clipping at 1.0; training took 2194 s on a CPU.

The drift stream is built from AG News [40], with four topic classes, and TeleQnA [41], a collection of telecommunication questions on standards and research with five categories, restricted to sentences of 4–30 words. An AG News document consists of a title and its description, and a TeleQnA document of a question, its explanation and its answer. The stream visits AG News, TeleQnA, AG News and TeleQnA for $\tau_{\mathrm{slow}} = 150$ steps each (600 steps), with $N_{\mathrm{c}} = 6$ parallel conversations, a topic change every $\tau_{\mathrm{med}} = 30$ steps and $\kappa = 0.9$; a change of domain ends every conversation. One run therefore carries 3600 messages of about 18 tokens. The stream draws its documents from the training split of each corpus, with replacement within a run, while the validation set of Section 5.2 is drawn from the validation split, so that no validation sentence occurs in the stream. The channel is AWGN with the SNR process (1) at $\mu_\gamma = 10$ dB, $\rho = 0.95$ and $\sigma_\gamma = 4$ dB, clipped to $[0,20]$ dB.

The context holds up to 48 tokens of the preceding messages of a conversation. The medium memory, consisting of the two fusion weights and $\mathbf{u} \in \mathbb{R}^{768}$, learns with Adam at a rate of 10^{-3} and is shipped every $C_{\mathrm{med}} = 10$ steps, at a cost of 6257 bits per shipment according to the ledger: 6256 bits for its 770 active entries and a 1-bit flag for an inactive cache-mixing weight that belongs to the same parameter group in the implementation. The slow memory has $K_{\mathrm{s}} = 2$ LoRA slots of rank $r = 8$ with $\alpha = 16$ on the query–key–value projection of the last six of the twelve layers, that is, 147,456 parameters per slot; it learns with Adam at 5×10^{-4} and is shipped every $C_{\mathrm{slow}} = 50$ steps at 1,180,032 bits per shipment. The slot is chosen every $C_{\mathrm{sel}} = 10$ steps with $\delta = 0.05$ bits per token, shipments use $b = 8$ bits per entry, and the adaptation loss (19) uses the prefix length of 6 symbols and one further prefix length per step. With these values, (23) gives $\bar{B}_{\mathrm{sync}} = 625.7 + 23{,}600.6 + 0.2 = 24{,}226.5$ bits per adaptation step, which coincides with the value recorded by the ledger. Table 2 collects the parameters.

5.2 | Compared Receivers and Decoder Selection

Three receivers are compared on the same streams, codebook and decoder: the *frozen* receiver, which decodes every message on its own with the unadapted prior; the *context memory* receiver, which adds the fast memory only and therefore requires no synchronization; and *DN-SC*, with all three memories. Each receiver is evaluated at every fixed prefix length and with selective refinement, and every relative change is stated against the frozen receiver at a fixed prefix, which needs neither signaling nor synchronization. Component ablations, namely single-rate shipping at approximately the same synchronization budget, elastic weight consolidation [34], shipping after every step, and DN-SC without slots, without context or with a single period for both shipped memories, stem from pilot runs of an earlier code version and are reported as exploratory in Section 6.4.

The decoder of the refinement policy was selected on validation text only, by a rule written down before the selection was run. The candidates comprised the first round (2, 4 or 6 symbols per token), the reopening of closed tokens (permitted or not) and the decoding of every pass (greedy, or beam search of width 2 or 4). The validation set contains one sentence, at a random position and with the true preceding sentences of its document as context, from each of 32 documents of the validation splits of AG News, TeleQnA and a public mirror of the News Category Dataset [42], that is, 96 messages and 1633 tokens, with the frozen prior, one AWGN realization per message and the SNR drawn from the stationary law of the SNR process (mean of the draws 9.8 dB). The criterion is the number of real symbols per token for 90% token accuracy under free-running decoding, with the signaling charged at its entropy. The first stage compares the six greedy candidates, counting candidates within 1% as tied and assigning a tie to the candidate with fewer rounds per message; the second stage adds beam widths 2 and 4 to the winner, and a beam is adopted only if it lowers the criterion by more than 2%, since it increases computation. Because a beam search changes the receiver, the fixed-prefix reference is decoded with the same beam.

5.3 | Evaluation Protocol and Statistical Analysis

Every message is scored prequentially, with the parameters that the receiver holds before the update of that step. Two measurements are taken. Teacher-forced emulation on every step scores each token given the true preceding tokens, which requires one pass of the language model per step and yields the complete operating curve over all prefix lengths and thresholds; the decisions of the emulated policy at $\tau = 0.9$ are written into the receiver's context. Real decoding on every 23rd step (26 steps and 156 messages per run; 23 is coprime with the shipping periods of 10 and 50 steps, so that the audited steps sample every phase of the shipping cycles) decodes the same received symbols free-running, round by round and with the configured decoder, for selective refinement at every threshold and for every fixed prefix alike. A decision rule fixed in advance determines which measurement constitutes the result: the teacher-forced curves are retained if, on the audited steps, real decoding requires at most 5% more forward symbols at the target than the emulation for every receiver of the headline table; otherwise the real-decoding figures replace them on both sides of every comparison. The implemented check covered the refinement policy of every receiver, whereas the rule as worded also covers the fixed-prefix receivers of that table, and applied as worded it selects the real-decoding figures (Section 6.3). The article nevertheless quotes the teacher-forced figures as the headline, because they are the more conservative, and reports the real-decoding figures alongside for every receiver; this is a declared deviation from the pre-specified rule.

Two simplifications of the stream simulation are stated explicitly. First, the adaptation loss (19) of a step reuses that step's channel realization; because an update reaches the receiver only after the step has been decoded and the noise is independent across steps, this is equivalent in distribution to a realization drawn by the transmitter for the update alone, although the shared realization couples the update weakly with the receiver's context of the next step. Second, every message is transmitted once with the gains (10) computed over all its tokens, and the refinement rounds, emulated and real alike, are read off that transmission, so that a refinement round carries its shell at the gain of the whole message rather than at the gain of the requested tokens. Section 6.3 quantifies the effect of the second simplification.

The frozen receiver and DN-SC were run on seeds 0–5 and the context memory receiver on seeds 0, 3, 4 and 5; its runs on seeds 1 and 2 were interrupted by a restart of the machine and not repeated. Seeds 3–5 had never been run before the protocol was fixed. Seeds 0–2 ran on a Linux server CPU with one thread per run and seeds 3–5 on a Windows 11 laptop CPU with two threads per run, from code revisions whose stream runner and decoder are identical. The seed is the unit of replication. Let d_j denote the relative change of a receiver against the reference on seed j of n_{s} seeds. The seed-level test is an exact sign-flip permutation test,

$$p = 2^{-n_{\mathrm{s}}} \sum_{\sigma \in \{-1,1\}^{n_{\mathrm{s}}}} \mathbb{1}\Big\{ \Big| \sum_{j=1}^{n_{\mathrm{s}}} \sigma_j d_j \Big| \geq \Big| \sum_{j=1}^{n_{\mathrm{s}}} d_j \Big| \Big\}, \tag{31}$$

whose smallest attainable two-sided value is $2/2^{n_{\mathrm{s}}}$, that is, 0.031 for six seeds, reached only when all seeds agree in sign, and 0.125 for four seeds. Over the n_{r} receivers compared with the reference, the ordered p-values are adjusted by the Holm procedure [43],

$$\tilde{p}_{(i)} = \max_{i' \leq i} \min\big\{1, (n_{\mathrm{r}} - i' + 1)\, p_{(i')}\big\}. \tag{32}$$

For real decoding, the operating curves of the audited steps are pooled over the seeds, every receiver is compared with the frozen receiver on the same seeds and steps, and the 95% interval of each relative change $\Delta = S(A^\star)/S^{\mathrm{ref}}(A^\star) - 1$ is the percentile interval of $N_{\mathrm{b}} = 2000$ paired bootstrap replicates $\beta = 1, \ldots, N_{\mathrm{b}}$ [44],

$$\Delta^{*(\beta)} = \frac{S^{*(\beta)}(A^\star)}{S^{\mathrm{ref},*(\beta)}(A^\star)} - 1, \qquad \mathrm{CI}_{95} = \Big[\Delta^*_{(0.025)}, \Delta^*_{(0.975)}\Big], \tag{33}$$

in which the audited steps are resampled in pairs within each seed and a replicate whose curve misses the target counts as an infinite cost. Differences in prior cross-entropy between receivers are given with 95% block-bootstrap intervals over steps with blocks of 25 steps [45], and a claim of no meaningful difference requires two one-sided tests [46] with the 90% interval inside ± 0.05 bits per token.

6 | Results and Discussion

This section first establishes the link-level behavior of the codec on the base corpus and the selection of the decoder on validation text. It then reports the primary endpoint on the drift stream, analyzes how the memories and the refinement policy contribute to it, and quantifies the cost of adaptation and of computation. A discussion of the scope and limitations of the findings concludes the section.

TABLE 3 | Free-running token accuracy and BLEU-4 on the base corpus (200 test sentences, AWGN, 10 dB, greedy decoding) for four receiver priors, against the number of real channel symbols per token m.

	Token accuracy				BLEU-4			
Receiver prior	$m = 4$	$m = 6$	$m = 8$	$m = 12$	$m = 4$	$m = 6$	$m = 8$	$m = 12$
None (maximum likelihood)	0.3728	0.7032	0.9119	0.9980	0.0354	0.3813	0.7672	0.9946
Unigram	0.4081	0.7366	0.9291	0.9985	0.0506	0.4261	0.8152	0.9971
GPT-2	0.4633	0.8570	0.9706	0.9995	0.1839	0.7059	0.9273	0.9990
GPT-2 + true history (diagnostic)	0.5510	0.8930	0.9818	0.9995	0.3103	0.7819	0.9607	0.9990

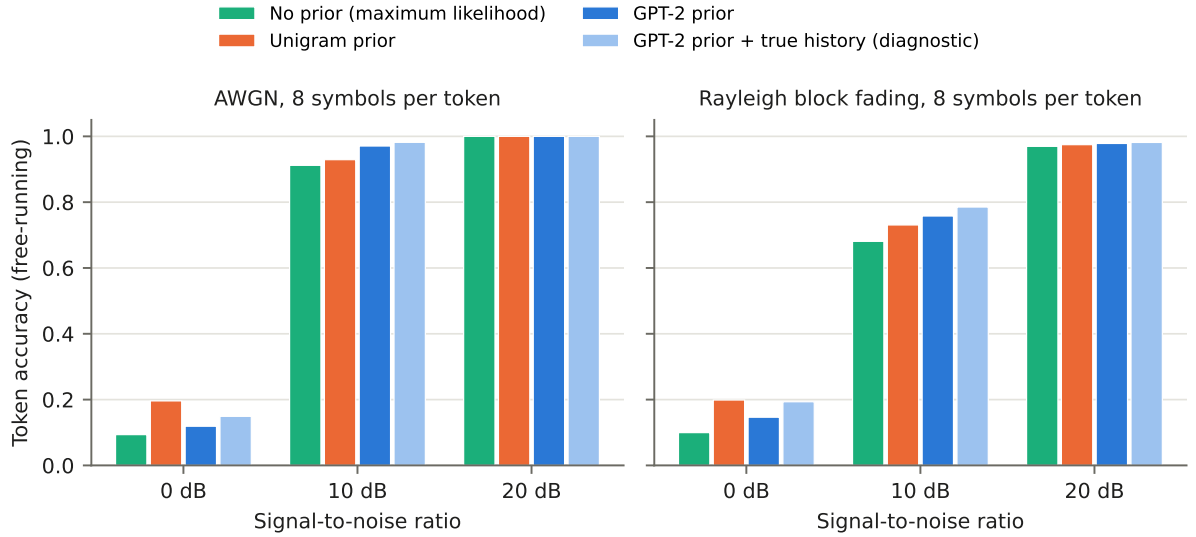


FIGURE 6 | Free-running token accuracy at 8 real channel symbols per token on the base corpus, by receiver prior, SNR and channel (greedy decoding).

6.1 | Link-Level Performance on the Base Corpus

Table 3 lists the free-running token accuracy and BLEU-4 of the codec on 200 test sentences of the base corpus over AWGN at 10 dB, for four receiver priors and a greedy decoder, and Figure 6 extends the comparison at 8 symbols per token to other SNRs and to Rayleigh block fading. The language prior lowers the rate required for a given fidelity at prefix lengths of 4 to 12 symbols; at 2 symbols per token GPT-2 is slightly below the other priors (0.110 against 0.127 without a prior and 0.138 with the unigram prior), and at 16 symbols every prior reaches a token accuracy of at least 0.9998. At 6 symbols per token, GPT-2 decides 85.7% of the tokens correctly, against 73.7% with token frequencies alone and 70.3% without a prior, and BLEU-4 rises from 0.38 to 0.71. The last row of the table is a diagnostic that provides the receiver with the true preceding sentences of the document and thereby bounds the contribution of the fast memory (89.3% at 6 symbols). Under Rayleigh block fading at 10 dB and 8 symbols per token, the four priors reach 0.681 (none), 0.731 (unigram), 0.758 (GPT-2) and 0.785 (GPT-2 with true history). At 0 dB, GPT-2 falls below the unigram prior under both channels at every prefix length (at 8 symbols per token, 0.119 against 0.196 over AWGN and 0.146 against 0.199 under Rayleigh fading), while the receiver without a prior remains the least accurate at 8 symbols (0.094 over AWGN), most likely because the greedy decoder then follows its own early errors. The operating point of this study is therefore an SNR of about 10 dB.

Rate nesting causes no measurable loss at the prefix length that was examined: a codebook trained

TABLE 4 | Selection of the decoder on validation text (96 messages, 1633 tokens, mean SNR 9.8 dB): real channel symbols per token for 90% token accuracy under free-running decoding. Selective refinement is charged its signaling at the entropy (criterion) or as a bitmap; the fixed prefix is decoded with the same beam width. Where a curve exceeds 90% at its lowest threshold ($\tau = 0.7$; all rows except the first and third), the value is read there and is an upper bound. Rounds per message at threshold $\tau = 0.95$.

Beam	First round (symbols)	Reopening	Forward only	+ signaling (entropy)	+ signaling (bitmap)	Fixed prefix, same beam	Rounds per message
1	2	no	7.527	8.971	9.442	10.994	5.00
1	2	yes	7.486	9.937	10.541	10.994	5.34
1	4	no	7.395	8.652	8.740	10.994	4.01
1	4	yes	7.682	9.640	10.060	10.994	4.22
1	6	no	7.836	8.730	8.758	10.994	3.01
1	6	yes	8.211	9.533	9.948	10.994	3.20
2	6	no	7.738	8.603	8.637	10.044	3.01
4	**6**	**no**	**7.492**	**8.306**	**8.357**	**9.049**	**3.01**

with the same budget for 8 symbols per token only reaches a channel-only token accuracy of 0.8890 at 10 dB on 2000 held-out tokens, whereas the nested codebook, read at its 8-symbol prefix, reaches 0.8885. Other prefix lengths were not examined. Before block power control, the codewords of the base corpus have an average power of 0.81–0.89 per symbol over the six prefixes, which block power control scales to exactly one per transmitted block.

6.2 | Decoder Selection

Table 4 reports the selection on validation text. In the first stage, a first round of 4 symbols yields the lowest criterion (8.652 symbols per token), and a first round of 6 symbols (8.730) lies within 1% of it and requires fewer rounds (3.01 against 4.01 per message), so that it is selected under the rule. Reopening is inferior in every case once its signaling is charged, because every token with shells left must then take part in every round's decision; at the lowest threshold its requests cost 2.3–4.2 bits per token at the entropy, against 1.5–2.5 without reopening. In the second stage, beam search of width 4 lowers the criterion by 4.9%, above the 2% bar, whereas width 2 (1.5%) does not. For six of the eight candidates, including the selected one, the operating curve already exceeds 90% at the lowest threshold of the validation grid ($\tau = 0.7$), so that the criterion is read there and constitutes an upper bound; the selected decoder reaches 94.5% at that point and the greedy decoder with the same first round 91.8%. The measured points nevertheless support the decisions: at their cheapest points, beam 4 is both cheaper and more accurate than the first-stage winner, and the bound of the 6-symbol first round can only decrease, so that this candidate ties with or outperforms the 4-symbol one.

The selected decoder, with beam width 4, a first round of 6 symbols and no reopening, requires 7.4% fewer symbols than the greedy decoder starting from 2 symbols. The selection also shows that beam search benefits the fixed prefix far more than refinement: with width 4, the fixed prefix requires 17.7% fewer symbols than with greedy decoding (9.049 against 10.994), because the beam removes much of the error propagation that refinement had previously compensated for. Every fixed-prefix reference in the remainder of this section is decoded with the same beam, so that all comparisons are made against this stronger baseline.

TABLE 5 | Real channel symbols per token for 90% token accuracy on the hierarchical drift stream (GPT-2 prior, beam-4 decoder for every receiver). Selective refinement includes its return-link signaling at the entropy; synchronization is not included. Teacher-forced: mean over seeds, all 600 steps. Free-running: 26 audited steps per seed, curves pooled over the seeds, 95% paired bootstrap interval in brackets. Changes are relative to the frozen fixed-rate receiver on the same seeds. Where a curve exceeds 90% at its cheapest point, the value is read there and is an upper bound (see text).

Receiver	Seeds	Teacher-forced	Change	Free-running	Change [95% CI]	p
Frozen, fixed rate	6	9.23	—	10.37	—	—
Frozen + refinement	6	7.80	−15.4%	8.36	−19.3% [−21.4, −15.6]	0.031
Context memory, fixed rate	4	8.04	−10.1%	9.05	−10.6% [−16.1, −5.3]	—
Context memory + refinement	4	7.18	−19.8%	7.55	−25.4% [−29.0, −15.6]	0.125
DN-SC memories, fixed rate	6	7.80	−15.5%	8.17	−21.2% [−23.9, −14.9]	—
DN-SC + refinement	6	**7.08**	**−23.3%**	**7.48**	**−27.9%** [−31.7, −21.6]	0.031

p: exact sign-flip test (31) on the per-seed costs under free-running decoding (refinement receivers against the frozen fixed-rate receiver); Holm-adjusted 0.094, 0.125 and 0.094 for the three rows. The context-memory receiver is compared with the frozen fixed-rate receiver on its own four seeds (0, 3, 4 and 5), on which the reference costs 8.95 symbols per token teacher-forced.

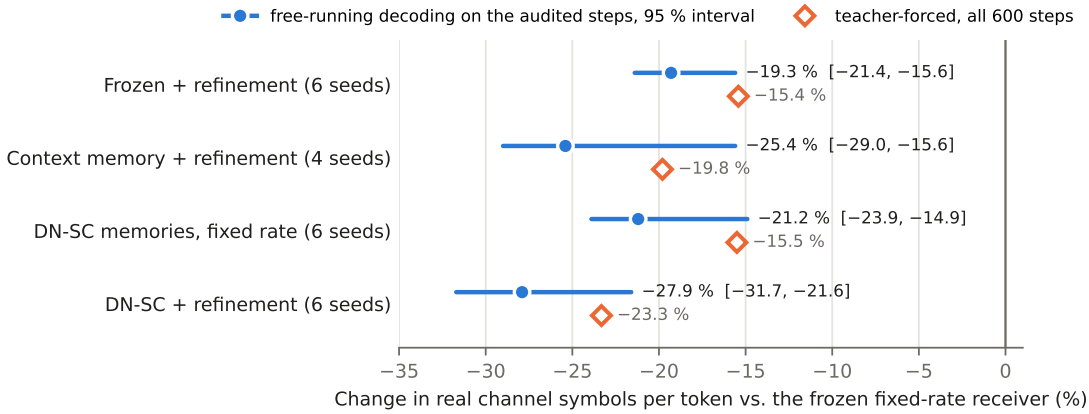


FIGURE 7 | Change in real channel symbols per token for 90% token accuracy against the frozen fixed-rate receiver, return-link signaling of refinement included. Dots: free-running decoding on the audited steps, with 95% paired bootstrap intervals; diamonds: teacher-forced on all 600 steps.

6.3 | Performance Under Semantic Drift

Table 5 and Figure 7 present the primary endpoint. With its memories and selective refinement, DN-SC reaches 90% token accuracy with 7.08 real channel symbols per token, compared with 9.23 for the frozen fixed-rate receiver. This is a reduction of 23.3% with the return-link signaling charged at its entropy, of 28.3% for the forward symbols alone and of 20.7% with an explicit bitmap. Figure 8 shows that the reduction occurs on every seed, between 10.5% and 28.5% (exact sign-flip test $p = 0.031$, the smallest value attainable with six seeds; Holm-adjusted over the three refinement receivers, $p = 0.094$), including the three seeds that had never been run before the protocol was fixed (20.6% as the ratio of the mean costs, 19.7% as the mean of the per-seed savings). Under free-running decoding on the audited steps, the reduction amounts to 27.9% (95% CI 21.6–31.7%; 26.3% with a bitmap), again on every seed, and to 25.9% (95% CI 11.0–33.2%) on the unseen seeds alone. Seed 3 is the weakest on both measures (−10.5% teacher-forced and −9.1% under free-running decoding). These figures exclude the synchronization of the shipped memories, which is charged in Section 6.5.

The reported savings of DN-SC are conservative estimates. Its operating curve already exceeds 90%

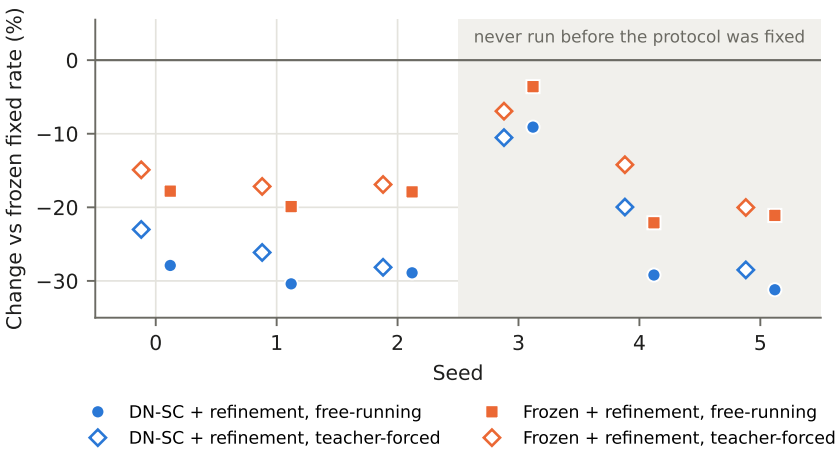


FIGURE 8 | Change against the frozen fixed-rate receiver on every seed, teacher-forced and under free-running decoding, for DN-SC and for the frozen receiver with refinement. Shaded: seeds that had never been run before the protocol was fixed.

at the cheapest point measured ($\tau = 0.5$) on four of the six seeds teacher-forced (90.3–93.1%) and on the pooled free-running curve (92.1%), so that its cost is read at that point, an upper bound of the cost at 90%. The same holds for the context memory receiver on three of its four seeds and on its pooled real-decoding curve (90.3%), and for the frozen receiver with refinement on one seed, whereas the frozen fixed-rate reference is always interpolated at 90%.

Applied to the refinement policy, as implemented, the decision rule of Section 5.3 retains the teacher-forced curves: on the audited steps, real decoding requires +3.4% (context memory), +3.6% (DN-SC) and +4.95% (frozen receiver) more forward symbols than the emulation. For the context memory and DN-SC, these are readings at the cheapest point, $\tau = 0.5$, at which real decoding is the more accurate of the two. The rule as worded also covers the fixed-prefix receivers, and for them it is not met: under free-running decoding, the fixed prefix requires +9.7% (frozen), +10.9% (context memory) and +3.8% (DN-SC) more symbols than emulated on the same steps (10.37 against 9.45 for the frozen reference). Applied as worded, the rule therefore selects the real-decoding figures, with a saving of 27.9% for DN-SC. Quoting the teacher-forced saving of 23.3% as the headline is a declared deviation from the pre-specified rule; it is made because a fixed prefix suffers more from error propagation than the refinement policy, so that the teacher-forced figures are the conservative ones, and both measurements are reported for every receiver. Figure 9 displays the pooled operating curves under free-running decoding. Refinement moves the curve of the frozen receiver to the left over the whole range it covers (87.7% accuracy and above), whereas the memories of DN-SC raise the fixed-prefix curve at every rate and thereby leave less to refinement.

The stream runs read every refinement round off a single transmission of the message (Section 5.3), so that a round carrying a subset of the tokens is sent at the gain of the whole message. To quantify the effect, the request policy was first emulated on 600 held-out sentences of AG News and TeleQnA (about 10,700 tokens), with the frozen prior, once without and once with the true preceding sentences as context, and once with that gain and once with every refinement round scaled to unit power over the tokens it carries, as in (10). With the gain of the whole message, the refinement rounds at $\tau = 0.9$ carry on average 1.15 (no context) and 1.12 (context) times the permitted power per symbol, because the requested tokens have

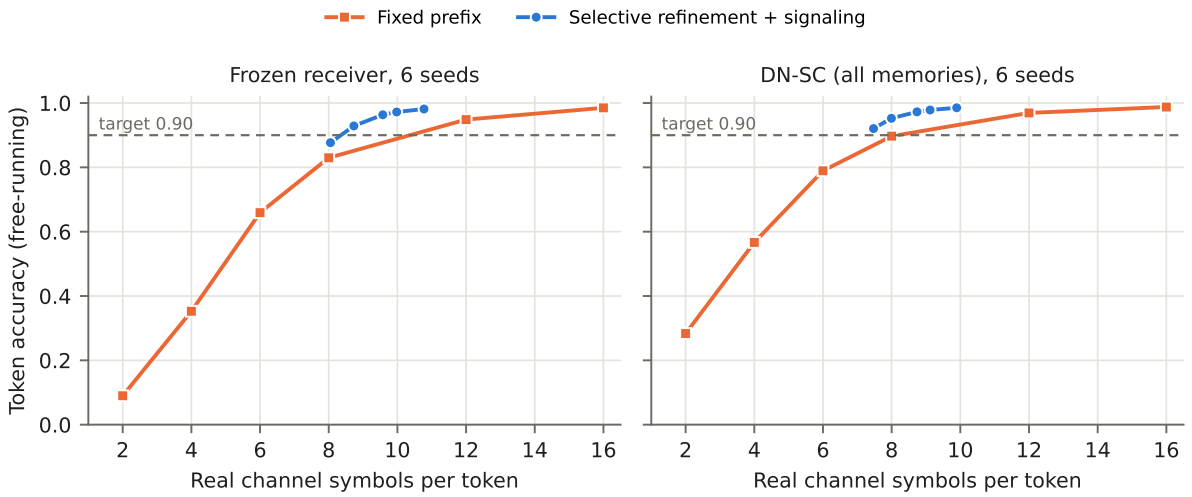


FIGURE 9 | Operating curves under free-running decoding on the audited steps of the drift stream, pooled over six seeds (156 steps, 936 messages per receiver). Fixed prefix: all six prefix lengths. Selective refinement: thresholds $\tau = 0.5$ to 0.99 from left to right, with the return-link signaling at its entropy added to the forward symbols.

more energy in the later shells than the average token; individual rounds range from about 0.5 to 2.0 (5th to 95th percentile). Transmitting them at unit power raises the cost of refinement at 90% token accuracy by 1.0% and 0.8%. Seed 0 of the stream was then repeated for the frozen receiver and DN-SC with every refinement round at unit power. The fixed prefix of the frozen receiver is unaffected (9.089 and 10.253 symbols per token, teacher-forced and under free-running decoding), and that of DN-SC changes by less than 0.1% through its context, which is written from the emulated policy. For the frozen receiver with refinement, the cost rises by 1.1%, and its saving falls from 14.9% to 13.9% teacher-forced and from 17.8% to 16.6% under free-running decoding. For DN-SC with refinement, which requests fewer shells because its prior is stronger, the cost rises by 0.4% (from 6.998 to 7.026 symbols per token), and its saving falls from 23.0% to 22.7% teacher-forced, while under free-running decoding it remains 27.9%. Judged by this seed, the six-seed savings of Table 5 overstate the effect of the complete system by about 0.3 percentage points and that of refinement on the frozen receiver by about one point; the conclusions are unaffected.

6.4 | Contribution of the Memories and of Refinement

The receiver memories and selective refinement both contribute to the saving, and their contributions overlap (Table 5). At a fixed rate, the memories of DN-SC alone save 15.5% teacher-forced (21.2% under free-running decoding) without any return-link signaling, and refinement alone saves the frozen receiver 15.4% (19.3% free-running). On top of the memories, refinement adds a further 9.3% teacher-forced and 8.4% free-running (95% CI 3.9–18.3%); since the refinement readings of DN-SC are upper bounds, these values are lower bounds of its contribution. A stronger prior leaves less uncertainty for refinement to resolve.

Table 6 exposes the mechanism. The prior cross-entropy (4) falls from 6.79 bits per token for the frozen receiver to 5.61 with the context memory and to 5.23 with all memories; on the four seeds shared by the three receivers, the context memory accounts for 76% of that reduction. The shipped memories lower the prior by a further 0.37 bits per token and the selective cost at $\tau = 0.9$ by 0.12 symbols per token.

TABLE 6 | Mechanism of the saving on the drift stream (teacher-forced, all 600 steps). Prior cross-entropy: mean ± SD over seeds; differences to DN-SC are paired per step, with 95% block-bootstrap intervals. Requests and rounds at the operating threshold $\tau = 0.9$.

Receiver	Seeds	Prior cross-entropy (bits/token)	Difference to DN-SC [95% CI]	Requests (bits/token) entropy / bitmap	Rounds per message	Synchronization (kbit/step)
Frozen	6	6.79 ± 0.04	$+1.56$ [$+1.41$, $+1.72$]	1.62 / 1.69	3.07	0
Context memory	4	5.61 ± 0.02	$+0.37$ [$+0.32$, $+0.41$]	1.40 / 1.53	2.81	0
DN-SC	6	5.23 ± 0.02	—	1.39 / 1.52	2.84	24.23

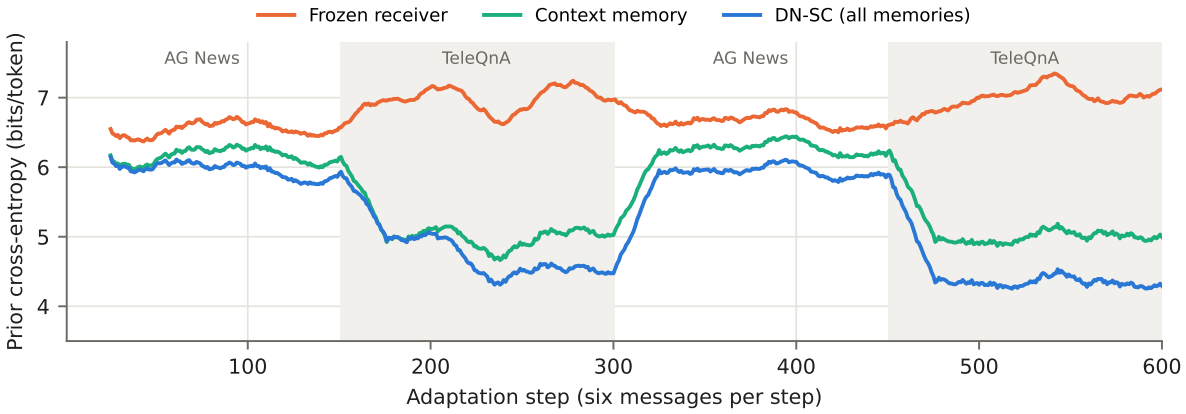


FIGURE 10 | Prior cross-entropy along the hierarchical drift stream (25-step moving average, mean over the four seeds shared by the three receivers). Shaded: TeleQnA regimes.

Both intervals exclude zero, although with four seeds the seed-level test cannot go below $p = 0.125$. The signaling remains light: at the operating threshold, the requests of DN-SC cost 1.39 bits per token at the entropy (1.52 as a bitmap) over 2.84 rounds per message, against 1.62 bits and 3.07 rounds for the frozen receiver, well below the bound of $K - k_0 = 3$ bits of Proposition 3.

Figure 10 follows the prior along the stream. Within AG News the three receivers differ by less than one bit per token. At the first switch to TeleQnA the frozen prior rises, whereas the context memory and DN-SC fall to about 5 bits per token within one topic period; a TeleQnA document, comprising question, explanation and answer, repeats much of its own vocabulary, which the context makes available to the prior. The shipped memories add their gain mainly within TeleQnA, and more so on its second visit, which is consistent with the recall of the adapter slot learned during the first visit. Figure 11 illustrates this behavior for one seed: the slow memory is shipped every 50 steps, in shipments of 1.18 Mbit each, the active slot follows the domain, and the slot learned on the first visit of TeleQnA is recalled on the second visit with the two-bit slot signal (five slot switches on this seed, including ten steps without an adapter, and 3.3 per run on average over the six seeds). Figure 12 shows where the requests occur under free-running decoding at $\tau = 0.9$: after the first round, the frozen receiver requests a further shell for about half of its tokens, whereas the context memory and DN-SC request one for about 37–38%. Of the tokens still open after rounds 2 and 3, all three receivers request a further shell for a similar fraction (about 53% and 34–37%), so that the memories act mainly by resolving more tokens in the first round.

Table 7 reports the components of the memories, obtained from pilot runs made with an earlier version of the code on a T4 GPU (greedy decoder, first round of 2 symbols, one to three seeds per receiver). These results are exploratory, and only the prior cross-entropy is reported, because it does not depend on the decoder of refinement except through the context. Without the context memory, the

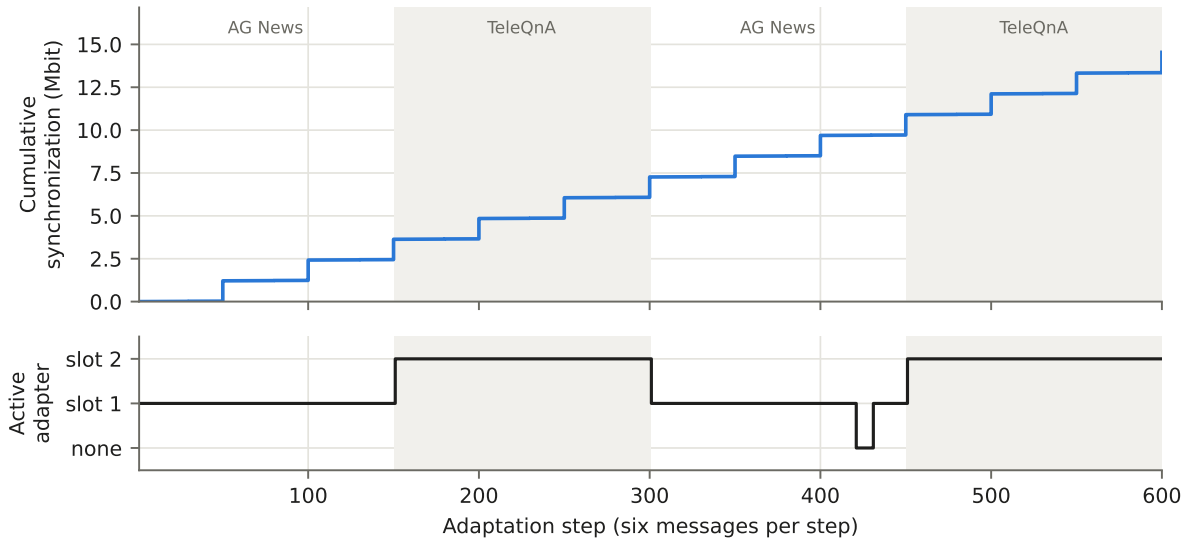


FIGURE 11 | Synchronization ledger and active adapter of DN-SC along the stream (seed 0). Top: cumulative synchronized bits, with a shipment of the medium memory every 10 steps and of the active adapter slot every 50 steps. Bottom: adapter slot chosen by (24); between steps 421 and 430 no adapter is active. The medium-memory shipments (6.3 kbit each) are too small to be visible at this scale.

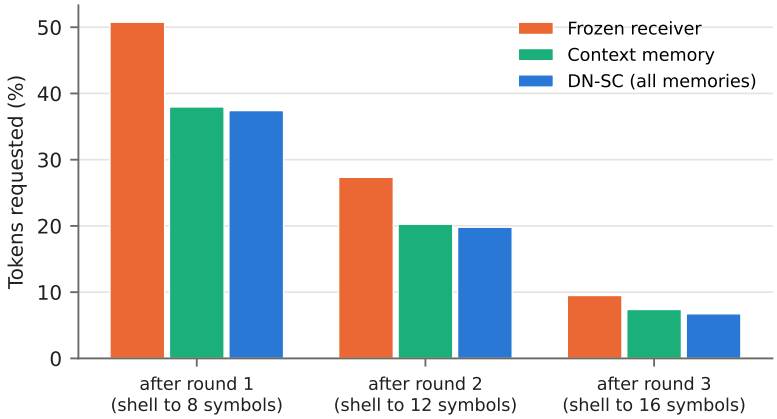


FIGURE 12 | Share of all tokens of the audited messages for which a further shell is requested after each round, under free-running decoding at $\tau = 0.9$, pooled over the audited steps of all seeds of each receiver.

prior cross-entropy of DN-SC rises by 1.14 bits per token (one seed). A single adapter shared by both domains, instead of one slot per domain, raises it by 0.10 bits per token. Shipping both gradient levels at one period with approximately the same synchronization budget (23.73 against 24.23 kbit per step) and a single adapter costs 0.11 bits, and adding elastic weight consolidation [34] does not help (0.12). The two separate shipping periods, by contrast, are not shown to matter: with slots and context retained and both levels shipped at one period, the difference is 0.02 bits per token, within the equivalence margin of ± 0.05, on one seed (at 29.63 against 24.23 kbit per step). Shipping after every step costs 49 times the air time of DN-SC and is equivalent to it within the same margin. On this stream, the gain of the shipped memories thus stems from the adapter slots and from infrequent shipping rather than from the separation of the timescales of the two shipped levels.

TABLE 7 | Component analysis (pilot runs, exploratory): prior cross-entropy on the drift stream, mean ± SD over seeds, paired per-step difference to DN-SC with 95% block-bootstrap interval, equivalence within ±0.05 bits per token by two one-sided tests (90% interval), and measured synchronization.

Receiver	Seeds	Prior cross-entropy (bits/token)	Difference to DN-SC [95% CI]	Equivalent (±0.05)	Synchronization (kbit/step)
Frozen	3	6.793 ± 0.044	$+1.537$ [$+1.315$, $+1.760$]	no	0
DN-SC without context	1	6.435	$+1.144$ [$+0.829$, $+1.443$]	no	24.23
Context memory	3	5.628 ± 0.023	$+0.372$ [$+0.322$, $+0.420$]	no	0
Single rate, one adapter	3	5.363 ± 0.024	$+0.107$ [$+0.076$, $+0.143$]	no	23.73
Single rate + EWC	1	5.414	$+0.123$ [$+0.073$, $+0.182$]	no	23.73
DN-SC without slots	3	5.352 ± 0.024	$+0.096$ [$+0.064$, $+0.135$]	no	24.23
DN-SC, one shipping period	1	5.312	$+0.022$ [$+0.013$, $+0.031$]	yes	29.63
DN-SC	3	$\mathbf{5.256 \pm 0.031}$	—	—	24.23
Shipping after every step	3	5.250 ± 0.031	-0.006 [-0.031, $+0.020$]	yes	1185.80

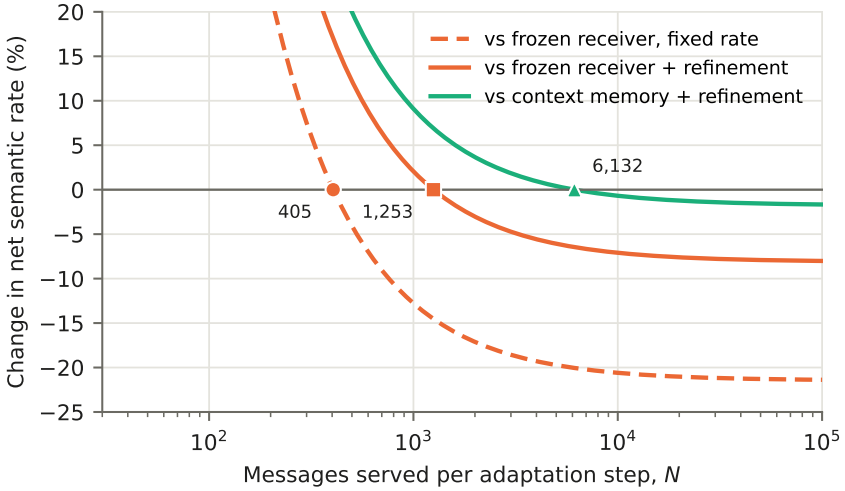


FIGURE 13 | Change in the net semantic rate of DN-SC, synchronization included, relative to three references, against the number of messages served by one adaptation step (four seeds shared by all receivers; return-link signaling at the entropy; synchronization charged at the capacity of 10 dB). Markers and labels: crossover traffic $N^\star$.

6.5 | Cost of Adaptation and Computational Complexity

Synchronizing the shipped memories requires 24.23 kbit per adaptation step, which at the capacity of 10 dB occupies about 14,000 real channel symbols, roughly 110 times the payload of one message at the target accuracy (about 127 symbols). Figure 13 shows the net semantic rate (28) of DN-SC relative to three references, on the four seeds shared by all receivers and with the signaling of refinement included. In accordance with Proposition 4, shipping pays against the frozen fixed-rate receiver once an update serves more than about 400 messages, and the saving then approaches 21.5%. Against the frozen receiver with refinement, the crossover lies at about 1250 messages (asymptote 8.1%), and against the context memory with refinement, which requires no synchronization, at about 6100 messages, with an asymptotic saving of only 1.8%. The last comparison is indicative only, because on these seeds both refinement readings are mostly upper bounds taken above 90% accuracy and are not taken at equal accuracy. In the stream itself, an update serves six messages. For a single link, the context memory with selective refinement is therefore the economical configuration, whereas shipped adaptation pays where one update serves many messages, such as the users of a cell or of a multicast group that share a domain.

The separation benchmark (30) relates these costs to the knowledge of the receiver. With the measured prior cross-entropies, it amounts to 3.93 real symbols per token for the frozen receiver and 3.02 for DN-SC, so that the memories lower the benchmark by 0.90 symbols per token. The measured costs exceed these benchmarks by factors of 2.35 (frozen receiver) and 2.58 (DN-SC memories) at a fixed rate, and of 1.99 and 2.34 with refinement; the benchmark is a lossless bound at the capacity of the mean SNR, whereas the measured costs are read at 90% token accuracy with uncoded token transmission. On the base corpus, an idealized separate design that compresses each message by arithmetic coding with the same GPT-2 model and transmits it with a finite-block-length channel code in the normal approximation [39] delivers 42% of the messages intact at 4 symbols per token and 96.5% at 6 symbols (AWGN, 10 dB, known SNR). This is a bound rather than a system, but it shows that the uncoded token-level codec does not outperform separation at a fixed rate and a known SNR (85.7% token accuracy at 6 symbols, Table 3). The potential advantages of DN-SC lie instead in a rate adapted per token by the receiver and in an explicit account of the cost of keeping the knowledge aligned; graceful degradation at an unknown SNR is not quantified here.

The receiver adds three small parameter sets to the frozen GPT-2 model (124,439,808 parameters): the codebook (213,248 parameters, trained once), the medium memory (771 values in the bit ledger, of which 770 are active in this configuration) and the adapter slots (147,456 parameters per slot). On a T4 GPU with one CPU thread, decoding one message of 19 tokens with the language prior took 193–199 ms with greedy decoding, depending on the prefix length, against 3.4–3.7 ms without a prior, so that the language model dominates the computational cost; the selected beam search of width 4 evaluates the language model for four hypotheses per position and was not timed separately. A complete 600-step run of DN-SC, including 26 audited steps decoded free-running at five thresholds and six prefix lengths with beam width 4, took 1.45–1.63 h on one thread of the Linux CPU, against 0.99–1.11 h for the frozen receiver. Because the transmitter must run the language model to learn the receiver's prior, adaptation costs transmitter computation in addition to air time.

6.6 | Discussion and Limitations

Every result in this section is read from files that the code writes and that are publicly available, under a protocol that was fixed before the confirmatory runs. Three findings hold under both the teacher-forced and the free-running measurement. A language prior with selective refinement saves channel symbols against a fixed-rate receiver with the same decoder, with the signaling included; the free context memory supplies most of what the memories gain; and shipped adaptation pays only above a traffic volume that a single link does not reach. The time nesting of the shipped memories is the weakest component, since the pilot ablation does not show that separating their periods matters and their measured benefit over the context memory is small.

The scope of the evidence is limited in several respects. Only one language model (GPT-2 with 124 million parameters) and one codebook were used; larger models will provide stronger priors at a higher cost of computation and of every adapter shipment. For the two six-seed refinement receivers, the exact test yields $p = 0.031$ against the frozen fixed-rate receiver (Holm-adjusted 0.094), while the context memory receiver has four seeds and cannot reach a value below 0.125. The channel is simulated (AWGN on the stream) with perfect channel state information and a known message length; signaling

and synchronization are charged at capacity and assumed error-free, and the delay of several rounds is counted but not modeled. On the stream, refinement and the receiver's context are emulated teacher-forced on every step and verified by real decoding on one step in 23, and the context is built from the emulated decisions. The six-seed stream runs transmitted refinement rounds at the gain of the whole message, which overstates the savings by up to about one percentage point as quantified above. The component analysis stems from pilot runs of an earlier code version with one to three seeds, and a stream with naturally occurring drift was not repeated with the final decoder. Finally, adapter slots are allocated greedily and never recycled, and a token is transmitted as an uncoded real vector without an outer channel code. The confirmatory runs were made on CPUs, and the pilot ablations and the latency measurement on a T4 GPU.

7 | Conclusion

This article presented doubly nested semantic communication, a token-level transceiver with a large-language-model prior that is nested in rate and in time and that charges every part of its air-interface overhead in channel symbols. The rate-nested codebook with per-shell block power control makes every prefix a valid transmission and token-level incremental redundancy exact, without measurable loss at the prefix length that was examined. A Bayesian beam-search receiver requests further symbols only for uncertain tokens, and three receiver memories adapt its prior at three timescales, with every synchronized bit recorded and converted into channel symbols.

On a hierarchical drift stream with six seeds and a decoder selected on validation text, DN-SC reached 90% token accuracy with 7.08 real channel symbols per token, against 9.23 for a frozen fixed-rate receiver with the same decoder. This is a reduction of 23.3% with the return-link signaling charged, observed on every seed, and of 27.9% (95% CI 21.6–31.7%) under free-running decoding of the audited messages. Most of the reduction in prior cross-entropy achieved by the memories stems from the conversation context, which requires no air time. The shipped memories lowered the prior by a further 0.37 bits per token but recovered their synchronization cost, relative to the context memory with refinement, only when one update served more than about 6100 messages, an indicative figure because both costs are upper bounds. For a single link, the context memory with selective refinement is thus the economical design, whereas shipped adaptation becomes worthwhile when one update serves a cell or a multicast group.

Future work will repeat the study with larger language models and on streams with naturally occurring drift. Signaling and synchronization will be carried by implemented codes over an error-prone return link, with the delay of the rounds modeled. One shipped update will be shared among the users of a cell, the regime in which the crossover analysis predicts that adaptation pays. Finally, the shipping period of each memory will be derived from its size and drift rate, so that the schedule of the memories follows from the cost model rather than from a design choice.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The code, the configurations, the pre-specified protocol, the decoder selection on validation text and every result file underlying the tables and figures of this article, including the per-step logs of all stream runs, are openly available at `https://github.com/abraheemrashid99-sys/Wiley_paper` (branch `review-fixes`). The script `scripts/paper_numbers.py` regenerates every number quoted in the article from these files. The corpora are publicly available.

References

[1] International Telecommunication Union, Radiocommunication Sector, "Framework and Overall Objectives of the Future Development of IMT for 2030 and Beyond," Recommendation ITU-R M.2160-0 (ITU, 2023).

[2] E. C. Strinati and S. Barbarossa, "6G Networks: Beyond Shannon Towards Semantic and Goal-Oriented Communications," *Computer Networks* 190 (2021): 107930.

[3] K. B. Letaief, W. Chen, Y. Shi, J. Zhang, and Y.-J. A. Zhang, "The Roadmap to 6G: AI Empowered Wireless Networks," *IEEE Communications Magazine* 57, no. 8 (2019): 84–90.

[4] W. Saad, M. Bennis, and M. Chen, "A Vision of 6G Wireless Systems: Applications, Trends, Technologies, and Open Research Problems," *IEEE Network* 34, no. 3 (2020): 134–142.

[5] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal* 27, no. 3 (1948): 379–423.

[6] W. Weaver, "Recent Contributions to the Mathematical Theory of Communication," in C. E. Shannon and W. Weaver, *The Mathematical Theory of Communication* (University of Illinois Press, 1949).

[7] D. Gündüz, Z. Qin, I. E. Aguerri, et al., "Beyond Transmitting Bits: Context, Semantics, and Task-Oriented Communications," *IEEE Journal on Selected Areas in Communications* 41, no. 1 (2023): 5–41.

[8] X. Luo, H.-H. Chen, and Q. Guo, "Semantic Communications: Overview, Open Issues, and Future Research Directions," *IEEE Wireless Communications* 29, no. 1 (2022): 210–219.

[9] M. Kountouris and N. Pappas, "Semantics-Empowered Communication for Networked Intelligent Systems," *IEEE Communications Magazine* 59, no. 6 (2021): 96–102.

[10] N. Farsad, M. Rao, and A. Goldsmith, "Deep Learning for Joint Source-Channel Coding of Text," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (IEEE, 2018), 2326–2330.

[11] H. Xie, Z. Qin, G. Y. Li, and B.-H. Juang, "Deep Learning Enabled Semantic Communication Systems," *IEEE Transactions on Signal Processing* 69 (2021): 2663–2675.

[12] E. Bourtsoulatze, D. Burth Kurka, and D. Gündüz, "Deep Joint Source-Channel Coding for Wireless Image Transmission," *IEEE Transactions on Cognitive Communications and Networking* 5, no. 3 (2019): 567–579.

[13] D. Burth Kurka and D. Gündüz, "Bandwidth-Agile Image Transmission With Deep Joint Source-Channel Coding," *IEEE Transactions on Wireless Communications* 20, no. 12 (2021): 8081–8095.

[14] Z. Wang, L. Zou, S. Wei, et al., "Large-Language-Model-Enabled Text Semantic Communication Systems," *Applied Sciences* 15, no. 13 (2025): 7227.

[15] L. Qiao, M. B. Mashhadi, Z. Gao, R. Tafazolli, M. Bennis, and D. Niyato, "Token Communications: A Large Model-Driven Framework for Cross-Modal Context-Aware Semantic Communications," *IEEE Wireless Communications* 32, no. 5 (2025): 80–88.

[16] S. Guo, Y. Wang, S. Li, and N. Saeed, "Semantic Importance-Aware Communications Using Pre-Trained Language Models," *IEEE Communications Letters* 27, no. 9 (2023): 2328–2332.

[17] F. Jiang, L. Dong, Y. Peng, et al., "Large AI Model Empowered Multimodal Semantic Communications," *IEEE Communications Magazine* 63, no. 1 (2025): 76–82.

[18] H. Xie, Z. Qin, X. Tao, and Z. Han, "Toward Intelligent Communications: Large Model Empowered Semantic Communications," *IEEE Communications Magazine* 63, no. 1 (2025): 69–75.

[19] L. Bariah, Q. Zhao, H. Zou, Y. Tian, F. Bader, and M. Debbah, "Large Generative AI Models for Telecom: The Next Big Thing?," *IEEE Communications Magazine* 62, no. 11 (2024): 84–90.

[20] H. Zhang, S. Shao, M. Tao, X. Bi, and K. B. Letaief, "Deep Learning-Enabled Semantic Communication Systems With Task-Unaware Transmitter and Dynamic Data," *IEEE Journal on Selected Areas in Communications* 41, no. 1 (2023): 170–185.

[21] Y. Zhang, J. Huang, J. Wang, J. Feng, and Z. Fei, "Knowledge Base Identification and Adaptation for End-to-End Semantic Communication," *IEEE Internet of Things Journal* 13, no. 18 (2026): 42698–42712.

[22] J. Zhang, S. Li, Y. Sun, et al., "Cooperative Semantic Knowledge Base Update for Semantic Communication Networks," *IEEE Transactions on Communications* 74 (2026): 1391–1405.

[23] G. Caire and D. Tuninetti, "The Throughput of Hybrid-ARQ Protocols for the Gaussian Collision Channel," *IEEE Transactions on Information Theory* 47, no. 5 (2001): 1971–1988.

[24] P. Jiang, C.-K. Wen, S. Jin, and G. Y. Li, "Deep Source-Channel Coding for Sentence Semantic Transmission With HARQ," *IEEE Transactions on Communications* 70, no. 8 (2022): 5225–5240.

[25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models Are Unsupervised Multitask Learners," OpenAI Technical Report (2019).

[26] E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations (ICLR)* (2022).

[27] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, "Nested Learning: The Illusion of Deep Learning Architectures," in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)* (2025), 52459–52493.

[28] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30* (2017), 5998–6008.

[29] P. Koehn, "Europarl: A Parallel Corpus for Statistical Machine Translation," in *Proceedings of Machine Translation Summit X: Papers* (2005), 79–86.

[30] M. Chen, Z. Sun, X. He, L. Wang, and A. Al-Dulaimi, "LLM-Based Semantic Communication: The Way From Task-Originated to General," *IEEE Wireless Communications Letters* 14, no. 10 (2025): 3029–3033.

[31] W. Ni, Z. Qin, Y. Feng, et al., "Multi-Task Semantic Communications via Large Models," *IEEE Communications Standards Magazine* 9, no. 4 (2025): 16–23.

[32] W. H. R. Equitz and T. M. Cover, "Successive Refinement of Information," *IEEE Transactions on Information Theory* 37, no. 2 (1991): 269–275.

[33] A. Kusupati, G. Bhatt, A. Rege, et al., "Matryoshka Representation Learning," in *Advances in Neural Information Processing Systems 35* (2022), 30233–30249.

[34] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al., "Overcoming Catastrophic Forgetting in Neural Networks," *Proceedings of the National Academy of Sciences of the United States of America* 114, no. 13 (2017): 3521–3526.

[35] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-Efficient SGD via Gradient Quantization and Encoding," in *Advances in Neural Information Processing Systems 30* (2017), 1709–1720.

[36] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-Efficient Learning of Deep Networks From Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, PMLR 54 (2017), 1273–1282.

[37] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* (2002), 311–318.

[38] J. D. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," in *Proceedings of the 18th International Conference on Machine Learning* (2001), 282–289.

[39] Y. Polyanskiy, H. V. Poor, and S. Verdú, "Channel Coding Rate in the Finite Blocklength Regime," *IEEE Transactions on Information Theory* 56, no. 5 (2010): 2307–2359.

[40] X. Zhang, J. Zhao, and Y. LeCun, "Character-Level Convolutional Networks for Text Classification," in *Advances in Neural Information Processing Systems 28* (2015), 649–657.

[41] A. Maatouk, F. Ayed, N. Piovesan, A. De Domenico, M. Debbah, and Z.-Q. Luo, "TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge," *IEEE Network* 40, no. 2 (2026): 253–260.

[42] R. Misra, "News Category Dataset," arXiv:2209.11429 (2022).

[43] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics* 6, no. 2 (1979): 65–70.

[44] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap* (Chapman & Hall, 1993).

[45] H. R. Künsch, "The Jackknife and the Bootstrap for General Stationary Observations," *Annals of Statistics* 17, no. 3 (1989): 1217–1241.

[46] D. J. Schuirmann, "A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics* 15, no. 6 (1987): 657–680.